

VDMJ Design Specification	
Author	Nick Battle
Date	05/10/26
Issue	2.10



0. Document Control

0.1. Table of Contents

0. Document Control.....	2
0.1. Table of Contents.....	2
0.2. References.....	2
0.3. Document History.....	3
0.4. Copyright.....	3
1. Overview.....	4
1.1. Package Overview.....	4
1.2. Class Mapping.....	6
2. Package Detail.....	7
2.1. vdmj.....	7
2.2. vdmj.lex.....	8
2.3. vdmj.syntax.....	11
2.4. vdmj.mapper.....	13
2.5. Analysis Packages.....	17
2.6. vdmj.typechecker.....	32
2.7. vdmj.pog.....	35
2.8. vdmj.runtime.....	42
2.9. vdmj.scheduler.....	48
2.10. vdmj.values.....	58
2.11. vdmj.plugins.....	61
2.12. vdmj.messages.....	63
2.13. vdmj.debug.....	64
2.14. vdmj.config.....	65
2.15. vdmj.util.....	67
3. External Interfaces.....	68
3.1. The Native Call Interface.....	68
3.2. The Remote Control Interface.....	69
3.3. The Remote Simulation Interface.....	70

0.2. References

- [1] Wikipedia entry for The Vienna Development Method,
http://en.wikipedia.org/wiki/Vienna_Development_Method
- [2] Wikipedia entry for Specification Languages,
http://en.wikipedia.org/wiki/Specification_language
- [3] The VDM Portal, <http://www.vdmportal.org/twiki/bin/view>
- [4] The VDMTools VDM-SL Language Manual,
http://www.vdmtools.jp/uploads/manuals/langmansl_a4E.pdf



- [5] The VDMTools VDM++ Language Manual, http://www.vdmttools.jp/uploads/manuals/langmanpp_a4E.pdf
- [6] DBGP - A common debugger protocol for languages and debugger UI communication, <http://xdebug.org/docs-dbgp.php>.
- [7] Overture - Open-source Tools for Formal Modelling, <http://www.overturetool.org/>.
- [8] Modelling and Validating Distributed Embedded Real-Time Control Systems, Marcel Verhoef, PhD Thesis.
- [9] Analysis Separation without Visitors. Proceedings of the 15th Overture Workshop: New Capabilities and Applications for Model-based Systems Engineering, Newcastle University, Sept 2017, Nick Battle.
- [10] <https://microsoft.github.io/language-server-protocol/overviews/lsp/overview/> and <https://microsoft.github.io/debug-adapter-protocol/overview>

0.3. Document History

Issue 0.1	22/10/08	First release.
Issue 0.2	28/11/08	Added comments from PGL. Added AST converter.
Issue 0.3	04/03/09	Added PO generator section, vdm.traces and misc other changes.
Issue 0.4	28/05/09	Added the DBGp protocol section, and extended runtime to discuss class initialization.
Issue 0.5	17/09/09	Added detail about VDM-RT implementation.
Issue 0.6	02/10/09	Updated CT description for TraceVariables
Issue 0.7	09/12/09	Added GPL copyright section.
Issue 0.8	22/01/10	Added section 3. Minor updates.
Issue 0.9	18/02/10	Various updates, and added section 4.
Issue 1.0	26/03/10	Updated for Overture 0.2.0, added new scheduling details
Issue 1.1	31/05/11	Updated for Overture 1.0.1
Issue 2.0	29/09/17	Updated for VDMJ 4.1.0
Issue 2.1	11/07/20	Added description of VDMJ Visitor framework
Issue 2.2	28/04/21	Updated DebugLink interface description and properties.
Issue 2.3	29/09/21	Added Catchpoint description.
Issue 2.4	14/11/21	Added description of mapper package.
Issue 2.5	27/03/22	Added ExternalFormatReader description.
Issue 2.6	04/09/22	Added IfdefProcessor description.
Issue 2.7	19/01/22	Updated various sections for visitors and missing classes.
Issue 2.8	07/02/23	Added vdmj.plugins package section.
Issue 2.9	12/12/25	Expanded the POG section to include operations
Issue 2.10	11/04/26	Added Remote Simulation section.

0.4. Copyright

Copyright © 2010-2026, Fujitsu Services Ltd.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts.



1. Overview

VDMJ provides tool support for the VDM-SL, VDM++ and VDM-RT specification languages, written in Java [4][5][8]. The tool includes a parser, a type checker, an interpreter, a debugger and a proof obligation generator. It is a command line tool only, though it is accessible from graphical environments like Eclipse [7] and VSCode [10].

1.1. Package Overview

The implementation is divided into the following Java packages, which are all sub-packages of `com.fujitsu`.

Packages	
<code>vdmj</code>	The main VDMJ class and supporting classes.
<code>vdmj.ast</code>	Classes that represent the output of the parser.
<code>vdmj.tc</code>	Classes that implement the type checker.
<code>vdmj.in</code>	Classes that implement the interpreter.
<code>vdmj.po</code>	Classes that implement the proof obligation generator.
<code>vdmj.mapper</code>	Classes that implement the ClassMapper.
<code>vdmj.lex</code>	Classes that implement the lexical analyser and its tokens.
<code>vdmj.syntax</code>	Classes that implement the syntax analyser.
<code>vdmj.typechecker</code>	Classes that support the type checker.
<code>vdm.pog</code>	Classes that support the proof obligation generator.
<code>vdmj.runtime</code>	Classes that support the interpreter.
<code>vdmj.scheduler</code>	Classes that implement the deterministic thread scheduler.
<code>vdmj.values</code>	Classes that represent runtime values in the interpreter.
<code>vdmj.commands</code>	Classes that read and execute commands from standard input.
<code>vdmj.plugins</code>	Classes that coordinate analyses using user-definable plugins.
<code>vdmj.messages</code>	Classes that hold VDMJ error and warning messages.
<code>vdmj.debug</code>	Classes that implement the debugger adapter interface.
<code>vdmj.config</code>	Classes that load and access property file settings.
<code>vdmj.util</code>	Utility classes and library routines used by all other packages.

Within the `ast`, `tc`, `in` and `po` sub-packages, there are further sub-packages as follows:

Packages	
<code>types</code>	Classes that represent types.
<code>expressions</code>	Classes that represent expressions.
<code>statements</code>	Classes that represent statements.
<code>patterns</code>	Classes that represent patterns and binds.
<code>traces</code>	Classes that represent trace definitions.
<code>definitions</code>	Classes that represent definitions.
<code>modules</code>	Classes that represent VDM-SL modules and their import/export definitions.

The `vdmj` package contains the “main” abstract class for the suite, with subclasses to parse, type check and interpret a specification in the VDM-SL, VDM++ or VDM-RT dialects.

The `vdmj.mapper` package contains classes that implement the mapping of AST parser classes to type checking classes, interpreter classes and POG classes.

The `vdmj.lex` package contains all classes to do with the lexical analysis of specifications. This includes the lexical token reader, plus a set of value classes for representing the various types of lexical token.

The `vdmj.syntax` package contains all the syntax analysis classes. This includes eight “readers”, which implement a backtracking recursive descent parser, based on a stream of lexical tokens. The readers are all sub-classes of an abstract `Reader` class.

The `vdmj.ast` contains immutable classes that hold the output of the parser. The `vdmj.tc`, `vdmj.in` and `vdmj.po` packages contain classes that implement the type checker, interpreter and proof obligation generator analyses. Each of these four packages have the following sub-packages:

- The `types` sub-package contains value classes that represent the various types that can be contained in a VDM specification. They are all sub-classes of an abstract `Type` class.
- The `expressions` sub-package contains value classes that represent the various types of expression that can be defined in a specification. They are all sub-classes of an abstract `Expression` class, which defines methods for an expression to be type checked and evaluated.
- Similarly, the `statements` package defines a set of value classes that subclass `Statement` and represent the different statements in a specification.
- The `patterns` sub-package includes value classes to represent the various patterns and binds in a specification. They are all subclasses of an abstract `Pattern` or `Bind` class.
- The `traces` sub-package includes classes that represent the possible trace definitions that can be declared.
- The `definitions` sub-package contains a set of classes representing the definitions in a specification. They are all sub-classes of an abstract `Definition` class. Definitions are nested, so for example a class definition may contain function definitions, which in turn may contain function definitions for their pre and post condition functions. All definitions implement methods to perform type checking, and to generate runtime values representing their content.
- The `modules` sub-package contains value classes that represent the modular structure of VDM-SL specifications, including their import and export declarations.

The `vdmj.typechecker` package includes classes which support the type checking of specifications. Most of the type checking is performed by the definition, expression and statement classes that collectively describe the specification under `vdmj.tc`, but the typechecker package defines the supporting classes to invoke the type checking methods of the other objects, and to represent the static environment in which type checking is performed.

The `vdmj.pog` package contains classes that support the proof obligation generator. Like type checking, the actual process of proof obligation generation is performed by the definitions, expressions and statements which form the specification under `vdmj.po`, but the `pog` package contains classes to represent proof obligations.

The `vdmj.runtime` package defines the abstract interpreter, and its subclasses to interpret specifications. It includes classes to represent the runtime execution context, and exceptions which can be generated at runtime.

The `vdmj.scheduler` package defines classes that schedule the deterministic execution of multiple threads, and for VDM-RT, coordinate the movement of time.

The `vdmj.values` class contains a set of classes to represent runtime values in the interpretation of a specification. They are all subclasses of the abstract `Value` class. All values are immutable, with the exception of the `UpdatableValue` class hierarchy, which has a `set` method and is used to implement all state variables in the system.

The `vdmj.commands` package contains the command line readers that implement the interactive actions of the suite.

The `vdmj.plugins` package contains an alternative, more flexible approach to reading the console and coordinating actions with various analyses. This is done via “analysis plugins” which perform arbitrary language actions and which can be provided by end users to extend the behaviour of the suite.

The `vdmj.messages` package contains values classes and exceptions for holding error and warning messages.

The `vdmj.debug` package contains classes which implement the debugger adapter interface. This allows arbitrary external programs to control a debug session. This is used by the Eclipse debugger in Overture via the DBGp protocol [6] and by the VSCode debugger via the LSP/DAP protocol [10].

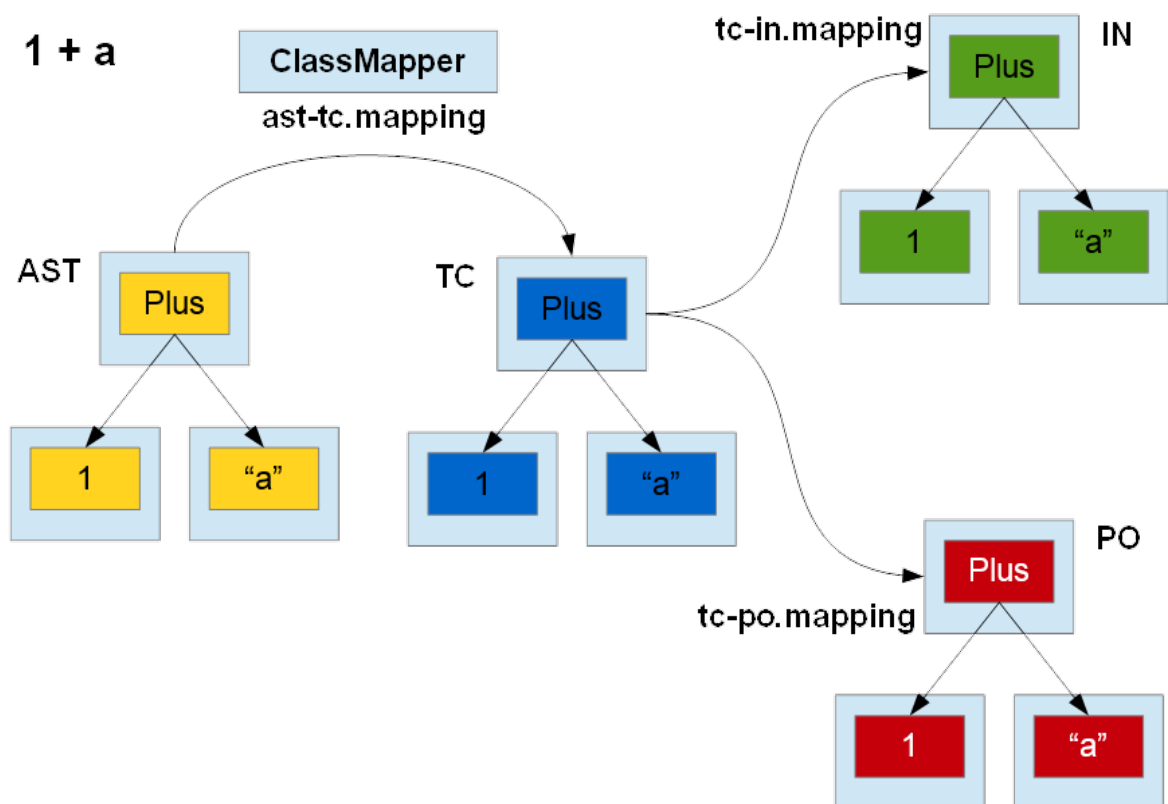
The `vdmj.config` package contains classes to access a properties file for reading various settings.

The `vdmj.utils` package contains common utilities used by all other packages.

1.2. Class Mapping

From VDMJ version 4 onwards, the analyses supported by the tool (ie. type checking, interpretation and proof obligation generation) have been separated into different packages. The AST output of the parser no longer contains the code to perform these analyses directly. To perform an analysis, the parser AST must first be converted to a different form, using classes that can perform that analysis. This process is called *class mapping* and is performed by the `ClassMapper`. See [9].

The process is driven by mapping files (`src/main/resources` files) which are a simple textual description of how to convert a class of one type into another type. These mapping files are used by the `ClassMapper`, and one `ClassMapper` instance is created for each analysis, with information describing how to create its tree of objects from either the AST or from another analysis. Mapping files are read by a `MappingReader` class.





2. Package Detail

2.1. vdmj

Class Summary	
VDMJ	The main class of the VDMJ parser/checker/interpreter.
VDMSL	The main class of the VDM-SL parser/checker/interpreter.
VDMPP	The main class of the VDM++ parser/checker/interpreter.
VDMRT	The main class of the VDM-RT parser/checker/interpreter.
Settings	A class with flags for -pre, -post, -inv, -dtc and -measures, as well as the dialect.
ExitStatus	An exit status code.

Enum Summary	
Release	An enumeration for the VDM language release: classic or VDM-10.

The vdmj package contains the “main” abstract VDMJ class for the suite, plus three concrete subclasses to parse, type check and interpret a specification in the VDM-SL, VDM++ or VDM-RT dialects. *These classes are now deprecated, in favour of vdmj.plugins.VDMJ. See 2.11.*

These classes just collect together a sequence of other classes to parse, type check and interpret the specification, depending on the command line arguments. The following (working) example illustrates the principles, using VDMJ classes to create a minimal interactive VDM-SL program:

```
public static void main(String[] args) throws Exception
{
    Settings.dialect = Dialect.VDM_SL;
    File file = new File(args[0]);
    LexTokenReader ltr = new LexTokenReader(file, Dialect.VDM_SL);
    ModuleReader mr = new ModuleReader(ltr);
    ASTModuleList modules = mr.readModules();

    if (mr.getErrorCount() == 0)
    {
        TCModuleList checked =
            ClassMapper.getInstance(TCNode.MAPPINGS)
                .init().convert(modules);

        TypeChecker tc = new ModuleTypeChecker(checked);
        tc.typeCheck();

        if (TypeChecker.getErrorCount() == 0)
        {
            INModuleList runnable =
                ClassMapper.getInstance(INNode.MAPPINGS)
                    .init().convert(checked);

            Interpreter interpreter =
                new ModuleInterpreter(runnable, checked);
            interpreter.init();

            CommandReader reader =
                new ModuleCommandReader(interpreter, "$ ");

            List<File> files = new Vector<File>();
```



```
        files.add(file);
        reader.run(files);
    }
}
```

The example would be almost exactly the same for VDM++ or VDM-RT with "Class" instead of "Module" in the various names (ClassReader, readClasses, ClassTypeChecker etc.), and the dialect constant changed from VDM_SL to VDM_PP or VDM_RT.

2.1.1. Comments

The structure of these classes isn't very flexible. In particular, if the "load" command is given to the CommandReader, it is awkward to recover if parse/type errors are discovered in the new set of filenames (the List<File> passed to the run method is updated by the command reader to pass the new file names back to be parsed and checked). The vdmj.plugins package considerably improves things.

2.2. vdmj.lex

Class Summary	
LatexStreamReader	A class to filter out LaTeX markup
IfdefProcessor	A class to filter out #ifdef/#else/#endif markers, using properties.
BacktrackInputReader	A class to allow checkpoints and backtracking while parsing a file.
LexToken	The abstract parent class for all lexical token types.
LexBooleanToken	A class to represent a boolean token.
LexCharacterToken	A class to represent a character literal token.
LexIdentifierToken	A class to represent an identifier.
LexIntegerToken	A class to represent an integer literal token.
LexKeywordToken	A class to represent keyword tokens.
LexLocation	A class to hold the location of a token.
LexNameList	A class to hold a list of LexNameTokens.
LexNameToken	A class to hold a name.
LexQuoteToken	A class to represent a quote type token.
LexRealToken	A class to represent a real literal token.
LexStringToken	A class to represent a string literal token.
LexTokenReader	The main lexical analyser class.
DocStreamReader	A class to read from Microsoft .doc files.
XMLStreamReader	An abstract class for reading XML structured documents.
DocxStreamReader	A class to read from Microsoft .docx files.
ODFStreamReader	A class to read from OASIS .odt files.
TextStreamReader	An abstract base class for text based ExternalFormatReaders
AsciiDocStreamReader	A class to read from AsciiDoc files.
MarkdownStreamReader	A class to read from Markdown files.
ExternalFormatReader	An interface implemented by other external format readers (eg. PDFs)



Enum Summary	
Dialect	An enumeration to indicate the VDM dialect being parsed.
Token	An enumeration for the basic token types.

Exception Summary	
LexException	An exception class for lexical analyser exceptions.

The `vdmj.lex` package contains all classes concerned with the lexical analysis of specifications. This includes the lexical token reader, plus a set of value classes for representing all types of lexical token.

The base class of the lexical system is `BacktrackInputReader`, which allows a stack of markers to be pushed within a stream of characters, returning the read pointer to the previous marker when a pop operation is performed. The class allows truly random movement within the stream – which is held as an array of Unicode chars, once it has been loaded.

The `BacktrackInputReader` opens a file with one of several stream readers: `LatexStreamReader`, `DocStreamReader`, `DocxStreamReader`, `ODFStreamReader`, `AsciiDocStreamReader` or `MarkdownStreamReader` – the choice is based on the file name, with the default being the `LatexStreamReader` (which also reads plain text). All of the stream readers implement `ExternalFormatReader`. Additional `ExternalFormatReaders` may be instantiated, to add other unsupported formats. This process is controlled by the `vdmj.parser.external_readers` property, which is a CSV list of `<suffix>=<class>` entries.

The input from all `ExternalFormatReaders` is processed by `IfdefProcessor` to apply `#ifdef` processing before the source is parsed. The `#ifdef` names available are the Dialect constants (eg. `#ifdef VDM_PP ... #else ... #endif`), plus any Java system properties. `Ifdef` statements must occur on a line by themselves. Line numbering is preserved by issuing blank lines for `#ifdef` suppressed sections.

The `LatexStreamReader` object strips out LaTeX markup and selects lines between `\begin{vdm_al}` and `\end{vdm_al}` markers, while preserving the line numbers.

The other stream readers search for markers, like “%% VDM %%” or “{vdm}” or “<!-- vdm -->”, in the file and extract the text between those markers, treating it as VDM source. This means that the line number information is lost, as this is much more complicated to work out in complex file formats. The abstract `XMLStreamReader` is common to .docx and .odt processing, and allows a ZIP file to be opened and a named subpart to be read as the source of the document. The abstract `TextStreamReader` is used for .adoc and .markdown files, and reads lines from a text file.

All formatting (like bold, italics and styles) within these documents is ignored by the stream readers. The XML formats work well with international character sets (as the character encoding is given in the XML) but the old MS .doc format may not work properly with non-ASCII encodings.

An `ExternalFormatReader` can implement the reading of a file however it wishes, but it is required to create a “`getText`” method that returns a character array for the extracted VDM content. The `vdmj.parser.external_readers` property is read by the `buildExternalReaders` method and associates file suffixes with `ExternalFormatReader` classes that will read them.

`LexTokenStream` extends `BacktrackInputReader`. Its purpose is to make the input file look like a continuous stream of `LexTokens`. The `nextToken` method returns the next token from the stream, and `getLast` will (repeatedly) return the last token read. Push and pop mark the stream and return to a mark respectively; unpush removes a marker without returning to it; the `retry` method does a pop followed by a push.

The `LexLocation` class is used throughout the system to represent a location within source code, for error message reporting and code coverage. The `toString` method of the class produces a string which can be appended to another message:

```
"in <class/module> (<filename>) at <line>:<column>"
```

All value objects in the system which have a sensible position in the source code have a `LexLocation`

associated with them. The class is also used to implement execution coverage tracking, with static methods to return the list of executable lines, and the locations hit or missed since the last time they were reset. It is also possible to merge coverage information from a file, written out by the `SourceFile` class (see section 2.13), which enables historical coverage to be managed.

There are several subclasses of the abstract `LexToken` to represent tokens that contain a value which is more naturally represented by a Java primitive type. For example, `LexRealToken` includes a double field, and `LexBooleanToken` contains a boolean.

The `Token` enumeration is the basic label for all token types. The enumeration includes a lookup method which, together with a `Dialect`, decides whether a given string is a token or not. Note that the dialect affects this: “static” is a token in VDM++ but is a legal variable name in VDM-SL, for example.

Lexical analysis throws a `LexException` if there are any problems. Recovery is left to the syntax analysis layer.

2.2.1. String Values

The source of a specification is represented by Java Strings internally, partly within lexical tokens and partly within classes that build on them, as covered in following sections.

But special care must be taken with values that represent VDM chars – that is, sequences of chars or individual chars. The parsing of these tokens accounts for the use of backslashed quoting for special characters. So the parser and subsequent processing has to understand whether these quoting characters are present in the data (eg. “\n” would be two characters) or not (“n” would be one newline character). This is particularly important in the `toString()` methods that all classes have.

The convention is that lexical classes have `toString` methods that give raw Java Strings, so “\n” would be a single newline character. Classes that come later, like the parser, type checker, interpreter and other analyses, have `toString` methods that produce *VDM source*. So an `ASTStringLiteralExpression`, which contains a parsed VDM string, would produce a double-quoted `toString` value, with any special characters within it quoted with a backslash.

Special care is needed in the runtime `Value` classes, in particular for `SeqValues`, which hold the runtime equivalent of string literals. If a `SeqValue` is seen to contain entirely `CharacterValues`, then its `toString` will produce a double-quoted and backslash quoted string value. If it contains any elements that are not `CharacterValues`, then it will revert to a standard “VDM sequence” output, with values surrounded by square brackets (ie. a VDM sequence enumeration).

In special circumstances, a runtime `Value` may need to be treated as a raw Java String. In that case, there is a `Value` method called `stringValue()`, which returns a Java String that does not have any quoting applied. The method is only implemented for `SeqValue` and `CharacterValue`.

2.2.2. Comments

There is some modest ugliness concerned with the parsing of certain symbol sequences that look like other tokens, eg. “mk_mod`name”. Naively, this would be a name (a `LexNameToken`) with a module part of “mk_mod” and the identifier part of “name”, but actually this is parsed as a single identifier, so that the syntax analyser can remove the “mk_” part and reveal the actual name.

Note that `LexLocations` have both start and end location information, though this is not used in VDMJ (it is used in the Overture editor though). The intention is to be able to identify blocks of source code that could be highlighted (eg. a whole statement or function, rather than just the start of one).

Recovering from a lexical error by raising an exception up to the syntax analysis may not be a good idea. The only recovery the syntax layer can do is to read up to some sort of safe point (eg. a semi-colon), and proceed from there. This gives comparatively poor error messages in general. It might be better to inject a likely token into the lexical stream, rather than throw an error and interrupt the stream.

`LexNameTokens` have a module/class name part and a simple name part (ie. they represent a grammatical *name* such as `C`xyz`). Whenever a name is created (as opposed to an identifier), it

therefore has to include its class or module name, even if none was actually used in the specification (eg. simple parameter names are identifier patterns that are characterized by a name token, so all parameter names are held as LexNameTokens like C`x). That would be fine, except that in VDM++ and VDM-RT the presence or absence of the class qualifier in a name can have semantic significance – explicitly identifying a member in a class hierarchy for example, with "object.X`name()". So LexNameTokens have an *explicit* flag, which means that the class name was explicitly specified by the caller, not implicitly filled in by the parser, and that flag is used during VDM++ type checking and runtime to identify the correct definition for the name. This works, but it may be over complicated. There are places where the *explicit* flag is set for very obscure reasons, which is a maintenance hazard. It may be better to take the Overture AST approach and allow names that simply don't have a class/module definition, and then take account of this when looking up definitions.

To enable VDM++ and VDM-RT function/operation overloading, LexNameTokens are mapped to TCNameTokens which optionally include a TCTypeList qualifier, representing the parameter types of the function or operation. The qualifier, if present, is used in the equals method of the class, and when searching for a name in a function/operation apply - the name sought is qualified with the actual types of the arguments. The equals function uses the TypeComparator to make the test, so a function declared with an int parameter will match one sought with a nat1 argument, etc. This system for managing overloading is not without its problems. Firstly, the use of the TypeComparator makes the equals method quite heavy. Secondly, it means that names cannot naively be used in Java maps because the hashCode of a function declaration name, and a function apply name may not be the same (if the argument types are not identical to the parameter types). Thirdly, it causes trouble when functions are applied via function variables (ie. lambda values) – such values can either be qualified with their parameter types or not, but since a function variable can either be applied (where argument types can be deduced) or just passed on (where they cannot), one or other of the name lookups will fail. To compensate, TCVariableExpression (which resolves simple names) will perform an unqualified "plain" name lookup if a qualified one fails. But a qualified lookup may succeed inappropriately by picking up an outer definition from the environment, when an inner unqualified name exists. This is tricky to solve.

2.3. vdmj.syntax

Class Summary	
SyntaxReader	The parent class of all syntax readers.
ExpressionReader	A syntax analyser to parse expressions.
TypeReader	A syntax analyser to parse type expressions.
DefinitionReader	A syntax analyser to parse definitions.
ClassReader	A syntax analyser to parse class definitions.
ModuleReader	A syntax analyser to parse modules.
PatternReader	A syntax analyser to parse pattern definitions.
BindReader	A syntax analyser to parse set and type binds.
StatementReader	A syntax analyser to parse statements.

Exception Summary	
ParserException	A syntax analyser exception.

The vdmj.syntax package contains all the syntax analysis classes. This includes eight "readers", which implement a backtracking recursive descent parser, based on a stream of lexical tokens. The readers are all sub-classes of an abstract Reader class.

Every SyntaxReader subclass is constructed by being passed a LexTokenReader object. The reader is then responsible for returning one or more syntactic elements that it is designed to parse. For example, an ExpressionReader can be attached to a lexical stream, and be used to read an

expression, or a comma separated expression list. Readers typically have methods called “read<something>” to perform the actual parse (eg. the minimal example above calls readModules from a ModuleReader).

Note that one type of reader usually needs other types of reader to complete its job. So for example, when a DefinitionReader is parsing an explicit function definition, it will use the raw lexical stream to read the function name, it will use a TypeReader to read the function’s type signature, a PatternReader to read the parameter patterns, and an ExpressionReader to read the body of the definition and any following pre or post conditions. All readers cache instances of the other readers used, and methods like getTypeReader either return the previous instance or create a new one. Note that there is no positional state information held in a reader therefore – each of them must depend on the LexTokenReader it references to (say) determine the last token read.

Several parts of the VDM grammar cannot be parsed unambiguously by reading lexical tokens in a strict sequence. For example, several parts of the grammar use a <pattern bind> symbol, which is defined to be either a pattern or a bind, but it is not possible to distinguish a pattern from a bind by looking at the next token in the stream – a type bind is a <pattern>:<type> for example, so it looks like a pattern to start with but turns out to be a bind. To overcome this, the parsers are able to backtrack. That is, the start of the <pattern bind> is marked (a push operation on the lexical stream), and an attempt is made to parse the “longest” possibility, which is a bind in this case; if that fails, the stream is popped back to the marker, and the other possibilities are tried. If all possibilities fail, there is clearly an error, though it is not clear which branch contains the error (eg. is it a pattern that is malformed or a bind that is malformed?). In this case, the parser reports the error from the branch which managed to consume the most tokens after the marker before failing. This is assumed to be the most helpful error, though it is not certain what the user intended.

The following backtrack code pattern is used frequently by the readers. Note how the ParserException is used to carry “depth” information about how far the parser progressed, and the two depths are compared at the end to see which exception to throw. The code calling this method may itself be backtracking, having pushed its own markers in the stream.

```
public PatternBind readPatternOrBind()
    throws ParserException, LexException
{
    ParserException bindError = null;

    try
    {
        reader.push();
        Bind b = readBind();
        reader.unpush();
        return new PatternBind(bind.location, b);
    }
    catch (ParserException e)
    {
        reader.pop();
        e.adjustDepth(reader.getTokensRead());
        bindError = e;
    }

    try
    {
        reader.push();
        Pattern p = getPatternReader().readPattern();
        reader.unpush();
        return new PatternBind(p.location, p);
    }
    catch (ParserException e)
    {
        reader.pop();
        e.adjustDepth(reader.getTokensRead());
        throw e.deeperThan(bindError) ? e : bindError;
    }
}
```



```
}  
}
```

The top level of the parsers contain the recovery code to attempt to move the parse beyond a given syntax error. This is done by each top level case (eg. parsing a whole function definition) defining two lists of tokens: those which should be read up to, and those that should be read up to *and past* in the event of an error. Then a common recovery method (“report” in the Reader class) reads tokens up to one of those specified before continuing with the parse. The objective is, say, for a Statement reader to read tokens up to the end of the broken statement before continuing. In general this is not foolproof, and often syntax errors produce a short cascade of unrelated errors later in the specification.

2.3.1. Comments

There are some ugly parts to the parsing. One is concerned with equals definitions, which are defined as “def” <pattern bind>=<expression> “in” <expression>, but if the <pattern bind> is actually a set bind of the form “e in set S”, this parses as “s in set (S = <expression>)”. There is some nifty footwork in the code to get round this (see the comments in readEqualsDefinition in the DefinitionReader).

Another ugly case is concerned with object call statements, which grammatically look like object apply designators as they end in ...<name>(args). So the parser reads an apply designator, then looks inside it to see whether the object being applied is a field designator or an identifier. The former is an object member invocation, the latter is a simple operation call.

The SyntaxReader base class provides a set of methods for reading and optionally advancing by one token. The differences between them are subtle (eg. advance and return the next token, or return the current token and advance), and I suspect the code could be cleaned up by reducing the number of options.

Recursive descent parsing always has difficulty with accurate error reporting and recovery. The method chosen is not perfect.

2.4. vdmj.mapper

Class Summary	
ClassMapper	Class to map one analysis tree into another
Mappable	An interface implemented by all objects that can be mapped
MappedObject	An abstract base class for mappable objects
MappedMap	An abstract base class for mappable Java Maps
MappedList	An abstract base class for mappable Java Lists
Mapping	A representation of one line in a *.mappings file
MappingReader	A parser to read *.mappings files.

2.4.1. Mapping Files

As described in 1.2, VDMJ version 4 and later separate the various analyses (type checking, PO generation and interpreting) from the AST classes.

A mapping between two analyses is defined in a *mappings* file, which consists of the following lines:

- Comment lines start with a ‘#’
- Package lines start with “package” and define the source and destination Java packages for the “map” lines that follow.



- Map lines start with “map” and define the fields of a source class that are passed to the constructor of a destination class.
- Unmapped lines start with “unmapped” and identify fully qualified source classes that are not mapped.
- “init” lines identify fully qualified static methods that are called during initialization.

So for example, the following lines are taken from the mapping file that converts the typechecker “TC” tree into the interpreter “IN” tree:

```
# Example mapping file entries - this line is a comment.  
init com.fujitsu.vdmj.in.annotations.INAnnotation.reset();  
package com.fujitsu.vdmj.tc.expressions to com.fujitsu.vdmj.in.expressions;  
map TCAndExpression{left, op, right} to INAndExpression(left, op, right);  
unmapped com.fujitsu.vdmj.lex.LexLocation;
```

Note that all lines, except comments, end in a semi-colon. Whitespace is ignored.

An “init” line indicates that a particular reset method should be called whenever the ClassMapper’s init method is called. This allows the target analysis to set things up – here, resetting something to do with annotations. Many init entries can be defined, and they are applied in order.

The example “package” line indicates that the following “map” lines are mapping members of the tc.expressions package to the in.expressions package. These lines just reduce the amount of clutter on the map lines. Many package lines can be used. Typically, the target tree has the same package structure as the source, so there will be a package line for each grammatical group.

The “map” line example describes how to turn a TCAndExpression into an INAndExpression. The source defines a set of fields (note the curly brackets) from the source class that are to be used; the target defines the *actual order* of those fields when passed to the target constructor. Therefore there must be a constructor which matches every map line in a mappings file. A special target field value of “this” indicates that the source should be passed unmapped to the constructor, even if elsewhere that same class is mapped. Map lines are required for every class that is mentioned in the target constructors, even if those classes are abstract. Abstract map entries don’t have to define any fields or parameters, e.g. “map TCDefinition{} to INDefinition(;)”. Map lines can additionally set fields using setter methods, after construction. Fields to be set are declared in the source class fields as usual, but the on the target side they appear after the constructor and the keyword “set”. For example:

```
map TCSomething{node, extra, also} to INSomething(node) set extra, also;
```

This example requires the INSomething class to have setter methods called setExtra and setAlso, that match the type of those fields.

Lastly, the “unmapped” line gives the fully qualified class name of something which will not be mapped, but rather used unchanged in the target tree. Here we see a LexLocation defined as unmapped, which is because there is no real difference in location information between TC and IN (or any analysis). Another common use of unmapped lines is for Java classes, like “unmapped java.lang.Boolean”.

If a target class (or any of its superclasses) has a field called *mappedFrom* with a type which can receive a value of the original source object, the field will automatically be set to the source in the conversion process. This enables (say) methods of an INSomething to directly access the TCSomething source object it was created from. This is occasionally useful, to interpret something simple from the source rather than create a complete mapping for it. Note that *mappedFrom* fields cannot subsequently be used as a source in “map” entries.

Mapping files are discovered on the Java classpath. So for example, they can be added to a jar, usually at the top level.

By convention, mapping files are named after the source and target packages that they define. For example, “ast-tc.mappings” and “tc-in.mappings”. You can see these files in src/main/resources, and as such they are packaged at the top level of the VDMJ jar.

When mappings files are loaded, every mapping file of the given name found on the classpath is

loaded. This allows plugins and annotation jars to include their own additional mappings for (say) tc-in.mappings.

Mapping files are parsed by a MappingReader, called via the ClassMapper's getInstance method, which is passed the name of the mapping file(s) to read. The ClassMapper maintains a cache of mapping files that it has read, so a subsequent call to getInstance with the same name will return the ClassMapper instance that deals with that one mapping. By convention, the name of the mapping file for the XX analysis is defined in XXNode, and called MAPPINGS. For example:

```
/**
 * The abstract root of all TC nodes.
 */
abstract public class TCNode extends MappedObject
{
    private static final long serialVersionUID = 1L;
    public final static String MAPPINGS = "ast-tc.mappings";
    ...
}
```

When a mapping file is read, the packages, classes and constructors are all checked and turned into the corresponding classes in the Java reflection package so that subsequent processing of trees is as efficient as possible. All mapped classes have to implement the Mappable interface (or be unmapped).

2.4.2. Mapping Collections

Often, a field in a source object will be some sort of Java collection – a List or a Map, for example. The basic Java classes do not implement the Mappable interface and so these classes cannot be mapped directly. Instead, these collections have to be treated as Mappable equivalents. To support this, the MappedList, MappedSet and MappedMap classes implement both Mappable and have constructors which convert the collection's content. For example:

```
# Map TCClassLists to INClassLists
map TCClassList{} to INClassList(this);

public class INClassList
    extends INMappedList<TCClassDefinition, INClassDefinition>
{
    public INClassList(TCClassList from) throws Exception
    {
        super(from);
    }
    ...
}

abstract public class INMappedList<FROM extends Mappable,
    TO extends Mappable> extends MappedList<FROM, TO>
{
    private static final long serialVersionUID = 1L;

    public INMappedList(List<FROM> from) throws Exception
    {
        super(INNode.MAPPINGS, from);
    }
    ...
}

abstract public class MappedList<FROM extends Mappable,
    TO extends Mappable> extends Vector<TO> implements Mappable
{
    private static final long serialVersionUID = 1L;

    @SuppressWarnings("unchecked")
    public MappedList(String mappings, List<FROM> from) throws Exception
    {

```



```
ClassMapper mapper = ClassMapper.getInstance(mappings);

for (FROM type: from)
{
    add((TO) mapper.convert(type));
}

...
```

So in this example, a list of `TCClassDefinitions` is converted from a `TCClassList` to an `INClassList`. The `INClassList` is constructed from a `TCClassList` and extends `INMappedList`, which in turn extends `MappedList`. The `INMappedList` is used by all lists in the IN tree and passes the `INNode.MAPPINGS` string (which is "tc-in.mappings") up to the `MappedList`, which uses the name to find the `ClassMapper` instance and then uses it to convert each of the source list objects.

The process for `MappedSet` and `MappedMap` is similar.

2.4.3. The Mapping Process

The mapping process proceeds as follows:

- Once the right mapping is identified, the `ClassMapper.getInstance(name)` method is called to either find and parse the mapping file(s), or to return the previously parsed data.
- The top level source object is passed to the `convert` method of the `ClassMapper` instance. This is very likely to be a collection, such as `TCModuleList` or `ASTClassList`.
- The `MappedList` constructor is passed the source collection, finds the `ClassMapper` instance created above, and loops through the content of the collection, converting each element.
- The mapper looks up the class of each element, and obtains the Java values of all of the fields identified in the mapping.
- Each field value is, in turn, converted using the `ClassMapper`.
- The converted fields are used to construct a target object, using the mapping.
- The target object is returned to the `MappedList`, which adds it to the target collection.
- Finally the `MappedList` returns the target list to the caller.
- The target tree can then be used to do whatever analysis it does.

There are two problems with a naive approach to converting classes like this: firstly, if a given object reference occurs in more than one place in the source tree, the target tree will create duplicates unless this is avoided; secondly, if a constructor argument is being converted, and the same object occurs deeper in the tree of "itself", there is a danger of infinite loops.

The first problem is solved by the `ClassMapper` keeping a map of source object IDs to target objects that have already been converted. If the same source object is converted, the previous target is returned. This preserves the duplications from the source tree. To support this, all mapped source classes must extend `MappedObject`. This includes a `getMappedId` method which returns a unique Java long for each instance. Typically, analyses define an `XXNode` that extends `MappedObject`, and that class also defines the name of the MAPPING file to use to create all of its subclasses.

The second problem is more subtle because until an object is created, you cannot predict its object reference. So when the conversion of each object is started, it is used to create a `Progress` object which is added to an "inProgress" Stack; each `Progress` object is popped when the conversion is completed. The private `Progress` class contains the source object being converted, and a list of `Pairs`, each of which defines another `Object` and a field name that references the source. When a set of constructor arguments are created, if any source fields are already in the `Progress` stack (ie. they are actively being converted), they are added to the `Progress` entry already on the stack. When a `Progress` entry is popped, the converted object reference is available and the fields that the `Progress` items lists are updated with the target.



2.4.4. ClassMapper Usage

Although the description above is quite complicated, the hope is that using the ClassMapper is as simple as reasonably possible.

If we assume that the mapping file for transforming XX classes into YY classes is complete and on the classpath, and the name of the mapping file is in the YYNode class that all YY classes extend, the process of converting an XX tree into a YY tree is as follows:

```
XXReader mr = new XXReader(source);
XXModuleList xxmodules = mr.readModules();
YYModuleList yymodules =
    ClassMapper.getInstance(YYNode.MAPPINGS).init().convert(xxmodules);
```

Here we assume that the XX tree is read from some source by an XXReader and that the result is a list of modules held in an XXModuleList. To convert this to a YYModuleList, the ClassMapper instance for the name of the XX-to-YY mapping is found/created by getInstance, and then it is initialized (since this is the start of a tree conversion). Finally the convert method is used to translate the XXModuleList into a YYModuleList.

A more realistic example is given in section 2.1, where an AST tree is converted to a TC tree for type checking and subsequently into an IN tree for the interpreter.

2.4.5. Comments

The mapping process may seem heavyweight, but it is surprisingly fast, as discussed in [9].

2.5. Analysis Packages

There are three analyses supported by VDMJ: type checking, interpretation and proof obligation generation. The implementation for these are contained in packages vdmj.tc, vdmj.in and vdmj.po respectively. Each of these packages has six sub-packages, which contain classes that perform the analysis related to one large group of AST node types.

The sub-packages are described below. Note that within each sub-package, classes related to the same AST node type have names prefixed with "TC", "IN" or "PO".

2.5.1. types

Class Summary	
(AST TC)Type	The parent class of all types.
***Type	A *** type. There are 25 such classes.
BasicType	The parent of the basic types (numbers, booleans and characters).
NumericType	The parent of the numeric types (real, rat, int, nat, nat1)
InvariantType	A type which has an invariant function associated with it.
NamedType	A type with a name.
OptionalType	An optional type.
ParameterType	A type associated with a polymorphic parameter name.
UnionType	A union of types.
UnknownType	A type representing a parser error.
UnresolvedType	A type name identifier by the syntax analyser.



VoidType	A type indicating the absence of a type.
VoidReturnType	A type indicating that a return statement has returned "()".
PatternListTypePair	A pattern list combined with a single type.
PatternTypePair	A pattern plus a type.
TypeList	A list of types.
TypeSet	A set of types.
TCTypeQualifier	A class to filter classes from a UnionType.

The types sub-package contains value classes that represent the various static types that can be contained in a VDM specification. They are all sub-classes of an abstract Type class.

Most simple types have a class dedicated to them of the same name, for example IntegerType or QuoteType. Similarly, the composite types have classes, like RecordType, SetType, SeqType and MapType; these have fields that in turn indicate the types of their components.

All TCTypes have a method to “resolve” themselves. The process of type resolution occurs early in the type checking process (see below), and turns TCUnresolvedTypes, which are just the names of types from the syntax analysis, into the actual type of the corresponding definition. The core of this happens in the typeResolve method of TCUnresolvedType, though other types call typeResolve recursively for any types that they contain – for example, TCFunctionTypes must resolve their parameter types and return type. The type resolution mechanism contains a recursive defence to avoid types which reference themselves from blowing the stack.

A static method called TCInstantiate.instantiate “polymorphs” types. This means that given a map of actual type parameters and names, it substitutes the actual type for any ParameterTypes they contain to yield a new Type object. This is used during the type check and execution of polymorphic function instantiations, when the actual type parameters are known.

All TCTypes implement a number of “is” and “get” methods – for example, (boolean) isMap and (MapType) getMap. These are used during type checking to determine whether a type is suitable for use in (say) a map application context. For simple types, these methods return false for the “is” method, except for the type concerned, which returns “true”; and “this” from the “get” method. For more complex types, these methods support the situation where the static type checking cannot know the actual runtime type, but knows that it is one of several. In this case – the most obvious example being a TCUnionType – the “is” method will return true if any of the member types of the union would return true; and the “get” method will construct a new type representing the aspects of the applicable members of the union, all spliced together. The type checking can then proceed using the information of this single blend of possibilities, in the knowledge that the tests it is making could occur at runtime.

For example, if a type is a union of two records, each of which has a field called “label”, one of which is a “seq of char” and the other of which is a “nat1”, the getRecord method of the TCUnionType would return a new synthetic TCRecordType with a single field called “label”, with type “(seq of char) | nat1”. So the type checking of access to the label field would proceed as though that was its type, even though at runtime the type will be one or the other.

A TCTypeQualifier class can be used with TCTypeUnion’s “match” method to return a TCTypeSet that only includes the types from the union that match, according to the qualifier. This is used during PO generation to check whether a subset of the types in a union are applicable, which will issue a subtype PO.

The INGetAllValuesVisitor, called via INTypeBind and INMultipleTypeBind is used during the evaluation of type binds. The visitor returns all the values for the type, though the only types which implements this are BooleanType, QuoteType and unions, records, sets or maps of these types; other types throw an exception if this is called, since they contain an infinite number of values.

TCUnknownTypes are used during error handling. Typically, an error will be encountered and reported, but rather than returning (say) the type of a sub-expression from type checking, a TCUnknownType is returned. This type has the property that all of its “is” methods return true, and its “get” methods will try to return a plausible Type. This means that subsequent type checking will not produce a cascade of errors as a single type checking fault deep in a specification winds its way out

to the top level.

The TCVoidType is usually used to mean the absence of a type, so for example an operation which returns “()” would be represented by a TCVoidType. During the type checking of a sequence of statements in an operation, any statements which follow a “return” statement on an execution branch will be unreachable (a warning). So to distinguish this deliberate return of nothing from most statements which yield nothing, the TCVoidReturnType is used.

The TCTypeList and TCTypeSet classes implement lists and sets of TCTypes, respectively. These are used in processing when a collection of types are encountered which must be turned into a single product type (TCTypeList) or a single union type (TCTypeSet). The TCProductType and TCUnionType classes contain one TCTypeList and TCTypeSet, respectively.

2.5.1.1. Comments

There is no TCReferenceType (compare with a ReferenceValue), yet several of the types do contain directly referenced types (like TCOptionalType and TCBracketType). It might be possible to simplify the hierarchy by adding one.

The type resolution process uses a mixture of eager and lazy evaluation. The main TypeChecker class (see below) calls the type resolution methods of the definitions in the specification, and these in turn resolve any TCTypes that they contain. But definitions do not recurse into the expressions and statements that they contain, even though they have unresolved TCTypes. Rather, unresolved types in expressions and statements are resolved later when the main type checking pass requires them.

Do we call them products or tuples? I went for TCProductTypes and TupleValues in the end.

2.5.2. expressions

Class Summary	
(AST TC IN PO)Expression	The parent class of all VDM expressions.
***Expression	An expression of type ***. There are >100 of these.
BinaryExpression	The parent of all binary expressions.
NumericBinaryExpression	The parent of all numeric binary expressions (+, -, *, /)
BooleanBinaryExpression	The parent of all boolean binary expressions (and, or, <=>, =>)
UnaryExpression	The parent of all unary expressions.
ExpressionList	A list of Expressions.

The expressions sub-package contains classes that represent the various types of expression that can be defined in a specification. They are all sub-classes of an abstract Expression class, which defines methods for an expression to be type checked, evaluated or PO generated.

The typeCheck method is implemented by all expressions, and is passed an environment defining the variables and types in scope, a list of argument types (for overloaded function and operation calls), the expected type of the result, and a NameScope which identifies what sorts of names are accessible (eg. whether state values are in scope – they are for expressions in operations, but not for expressions in functions).

The typeCheck method returns a TCType which indicates the result of analysing the expression in the environment passed. So literal expressions simply return an appropriate type, like TCIntegerType. More complex expressions have to consider whether the definitions they contain affect the environment, whether those definitions have to be type checked, and whether the type check of any sub-expressions return the expected result for the overall expression.

For example, consider a “forall” expression. This will contain a bind list of variables, and a predicate to evaluate for each (at runtime). The typeCheck method of TCForAllExpression is as follows:

```
@Override
public TCType typeCheck(Environment base, TypeList qualifiers,
    NameScope scope, TCType constraint)
{
    Definition def = new TCMultiBindListDefinition(location, bindList);
    def.typeCheck(base, scope);
    Environment local = new FlatCheckedEnvironment(def, base);

    if (!predicate.typeCheck(
        local, null, scope, new TCBooleanType()).isType(TCBooleanType))
    {
        predicate.report("Predicate is not boolean");
    }

    local.unusedCheck();
    return checkConstraint(constraint, new TCBooleanType());
}
```

A `TCMultiBindListDefinition` is a type of `Definition` (see below) which, when given the bind list for the forall expression, can expand the `Environment` passed in to make the names and types of the bind variables visible. Once created, the new definition is type checked to make sure the bindings contain no errors (for example, if the bind list contains a set bind, the expression representing the set must be a `TCSetType`).

A new `FlatCheckedEnvironment` is created to chain the new definitions onto the base `Environment` passed in, and this is used to type check the predicate of the forall expression. The return value of this is the `Type` of the predicate, which must be a boolean expression. The local environment is then checked to see whether all the names added to it by the bind list were actually used when type checking the predicate; any unused variables generate a warning. Lastly, this method returns a boolean `Type`, since a forall expression returns a boolean, after checking that this meets the constraint.

The `typeCheck` method on `TCEExpression` is also passed a `TCTypeList`. This is used when trying to resolve name overloading during function and operation application in VDM++ and VDM-RT. The `typeCheck` method of `TCApplExpression` starts by generating a `TCTypeList` from the `typeCheck` of each of the argument expressions it has. That may generate (say) `[int, int, bool]`. That list is then passed to the `typeCheck` method for the root of the apply (the thing being applied). If this root is a `TCVariableExpression` or a `TCFieldExpression`, the name of the variable or field is qualified with the list of types passed in, and this is used to find an overloaded name of a function or operation definition that has parameters whose types are compatible with the arguments (note, compatible with, not identical to). If it turns out that the variable or field is actually a map (which has no qualifiers), the search is repeated for the name without type qualification.

The other important method on `INExpressions` is the `eval` method. This is called to evaluate the expression given the runtime `Context` (the runtime equivalent of an `Environment`). The method returns a `Value` object, which can represent any value in VDM. The `eval` method for `INPlusExpression` is as follows:

```
@Override
public Value eval(Context ctxt)
{
    breakpoint.check(location, ctxt);

    try
    {
        Value l = left.eval(ctxt);
        Value r = right.eval(ctxt);

        if (NumericValue.areIntegers(l, r))
        {
            long lv = l.intValue(ctxt);
            long rv = r.intValue(ctxt);
        }
    }
}
```

```
        long sum = addExact(lv, rv, ctxt);
        return NumericValue.valueOf(sum, ctxt);
    }
    else
    {
        double lv = l.realValue(ctxt);
        double rv = r.realValue(ctxt);
        return NumericValue.valueOf(lv + rv, ctxt);
    }
}
catch (ValueException e)
{
    return abort(e);
}
}
```

All INExpressions and INStatements contain a Breakpoint object, and the eval method of all expressions and statements call their breakpoint's check method at the start. This usually does nothing, unless a breakpoint is set at this location, in which case execution stops and calls debugger code.

The INPlusExpression evaluates its left and right hand sides, and converts the resulting Value objects to raw Java longs or doubles. The result is a new Value, created using the valueOf method of NumericValue, which will create the simplest type of NumericValue capable of holding the result of the addition – for example, this might be an IntegerValue (for -123) or a NaturalOneValue (for +123) or a RealValue (for 1.23).

Note that the code may throw a ValueException, and that this is caught within the eval method rather than being propagated. This can only occur in the conversion of the sub-expression results to doubles, or the construction of the result Value. ValueExceptions indicate problems while evaluating an expression, but they are caught and propagated using the abort method, which creates and throws a ContextException (a Java RuntimeException). This is done to distinguish between expected value errors – for example, trying to convert a Value to one of several types in a union, and failing before the right one is found – and serious errors, which should cause the system to halt. The value system does not have location information (what is the location of the pure value 123?), and so this double-propagation of the exception also allows the value error to be located in an expression that caused it.

The most complex evaluations are for functions and operations, but these are delegated to the corresponding FunctionValue and OperationValue classes. This is so that a function value can be separately created (for example via a lambda expression) and applied. Operations cannot be used like this, but having operation values too means that, in VDM++ or VDM-RT, an object becomes a map of names to values – whether those are instance variable values or member operations and functions. The eval method of the INApplyExpression to call a function is therefore just:

```
try
{
    Value object = root.eval(ctxt).deref();

    if (object instanceof FunctionValue)
    {
        ValueList argvals = new ValueList();

        for (Expression arg: args)
        {
            argvals.add(arg.eval(ctxt));
        }

        FunctionValue fv = object.functionValue(ctxt);
        return fv.eval(argvals, ctxt);
    }
    else if (object instanceof OperationValue)
    ...
}
```



All INExpressions implement a method called findExpression. This has a line number parameter, and all implementations are responsible for returning themselves if they start on the line number, or recursing into their sub-expressions if they have them. This is used to set breakpoints on specific lines of a specification.

2.5.2.1. Comments

The implementation of name overloading may be problematic. A LexNameToken is optionally qualified with a TCTypeList, and the equals method uses the TypeComparator (part of vdmj.typechecker) to make a compatible comparison of the two names' type lists. Care must be taken when comparing names, especially during the "bootstrap" phases when qualifiers are not yet available.

2.5.3. statements

Class Summary	
(AST TC IN PO)Statement	The parent class of all statements.
***Statement	A statement of type ***. There are 40 or so of these.

The statements sub-package contains value classes that represent the various types of statement that can be defined in a specification. They are all sub-classes of an abstract Statement class, which defines methods for a statement to be type checked and executed.

Many of the principles for Statements are the same as those for Expressions covered above. Like Expressions, all Statements include a typeCheck method which is passed an Environment, a NameScope and a result constraint – though note that there is no need for a TCTypeList of qualifiers because an operation call can only be rooted on something that is already known by the statement (an operation name or an object designator), whereas function application in an expression has to evaluate an arbitrary expression to generate the root to which to apply the arguments.

Similarly, like INExpressions, all INStatements define an eval method which executes them, returning a Value – though statement executions usually return VoidValues.

TCStatements implement a method called exitCheck, which explores the statement tree (following blocks and branches from compound statements, using a visitor) looking for statements which can raise an exit status, like the TCExitStatement itself. This is used in the type checking of statements that catch and process exits, like TCTrapStatements. The method returns a set of exit TCTypes that can be thrown.

All INStatements implement a method called findStatement. This has a line number parameter, and all implementations are responsible for returning themselves if they start on the line number, or recursing into their sub-statements if they have them. This is used to set breakpoints on specific lines of a specification.

2.5.3.1. Comments

The exitCheck is not perfect. The detail is in the TCExitChecker visitor. This may produce false negatives (indicating that operations can't exit, when in fact they can). I gather VDMTools is better, but not perfect either. It is known to produce false positives.

2.5.4. patterns

Class Summary	
(AST TC IN PO)Pattern	The parent type of all patterns.



****Pattern	A pattern of type ****. There are 14 such pattern types.
Bind	The parent class of SetBind and TypeBind.
MultipleBind	The parent class of MultipleSetBind and MultipleTypeBind.
PatternBind	A pattern or a bind.
PatternList	A list of patterns.

The patterns sub-package includes value classes to represent the various patterns and binds in a specification. They are all subclasses of an abstract Pattern, Bind or MultipleBind class.

Patterns generate definitions, given a TCType. For example, the pattern “[a,b,c]” will produce definitions for the three integer variables, given that it is of type “seq of int”. The process of definition generation is recursive over the tree of nested patterns that might be defined. So the `getDefinitions(TCType)` method which all TCPatterns implement, uses a visitor to recurse for those pattern types which are defined as containing sub-patterns. The leaves of the pattern tree are the simple pattern types: TCBooleanPattern, TCCharacterPattern, etc. At the leaves, it is only TCIdentifierPattern which produces variable definitions.

Similarly, INPatterns generate a list of name/value pairs given a Value. The value is matched against the “shape” of the pattern and its sub-patterns, and any INIdentifierPatterns at the leaves are populated with the corresponding part of the original value. This `getNamedValues` method is called from definitions which include patterns (such as function parameter definitions) when the actual values are required.

INPatterns can also be asked for a simple list of their variable names, which involves a depth search for INIdentifierPatterns using a visitor.

TCPatterns include a `typeResolve` method because TCExpressionPatterns can contain references to TCUnresolvedTypes that need to be resolved early in the type check. TCDefinitions which include TCPatterns recurse into the pattern's `typeResolve` from their own `typeResolve` methods.

An INBind, which is sub-classed by INSetBind and INTypeBind, comprises a INPattern and a set of Values (either an explicit set, or the set of all the Values that a TCType can generate, in theory). These are used in quantified expressions, like “exists {a,b} in set S & a.type = b.type”. Here the set pattern {a,b} contains two identifier patterns that must (potentially) iterate through all the elements of S, taking the corresponding values. To drive the iteration, the INBind needs to generate all the possible Values, which are then given to the pattern to generate name/value pairs for each iteration. Therefore INBind has a method called `getAllValues`, which is implemented by both sub-classes (the TypeBind implementation calls the `getAllValues` method of the TCType, which is an error for everything except BooleanType, QuoteType and unions, records, maps and sets of these).

INMultipleBind, which is sub-classed by INMultipleSetBind and INMultipleTypeBind, is very similar except that they comprise a list of patterns and a set or type. But they still have a `getAllValues` method which collects together all the possible values in the set.

Note that in VDMJ, the generation of all values from a set *could* include the permutations of all the orderings of the values in that set. Internally, sets of Values are held in a ValueSet which is actually an ordered list (but with set semantics, with regard to no duplicates). Therefore the `getAllValues` methods for the various set binds could call the `permuteSets` method of ValueSet (via the same method on SetValue, which preserves the ordering). Note also that the original set is sorted before this process, which means that given the same set content, the order of processing in a bind (and therefore any looseness based on it) is deterministic. However, it is simpler, and legitimate *not* to permute sets when performing binds, and just depend on the initial ordering of the set.

```
@Override
public ValueList getBindValues(Context ctxt)
{
    try
    {
        ValueList results = new ValueList();
        ValueSet elements = set.eval(ctxt).setValue(ctxt);
        elements.sort();
    }
}
```



```
        for (Value e: elements)
        {
            e = e.deref();

            if (e instanceof SetValue && permuted) // optional
            {
                SetValue sv = (SetValue)e;
                results.addAll(sv.permutedSets());
            }
            else
            {
                results.add(e);
            }
        }

        return results;
    }
    catch (ValueException ex)
    {
        abort(ex.getMessage(), ctxt);
        return null;
    }
}
```

2.5.4.1. Comments

Binds and MultipleBinds look like they have a lot in common and could probably be put together to avoid a small amount of code duplication.

2.5.5. traces

Class Summary	
(AST TC IN PO)TraceDefinition	An abstract class representing a trace definition.
TraceDefinitionTerm	A class representing a sequence of trace definitions.
TraceLetBeStBinding	A class representing a let-be-st trace binding.
TraceLetDefBinding	A class representing a let-definition trace binding.
TraceRepeatDefinition	A class representing a repeated trace definition.
TraceCoreDefinition	Abstract of all core trace expressions.
TraceApplyExpression	A class representing a core trace apply expression.
TraceBracketedExpression	A class representing a core trace bracketed expression.
TraceConcurrentExpression	A class representing a core non-deterministic expression.
TraceIterator	An abstract class representing an expansion node.
AlternativeIterator	An expansion node for alternatives.
RepeatIterator	An expansion node for repeats.
SequenceIterator	An expansion node for sequences.
StatementIterator	An expansion node (leaf) for statement applies.
ConcurrentIterator	An expansion node for non-deterministic sequences.
TestSequence	A sequence of CallSequences
CallSequence	A sequence of Statements.
PermuteArray	A utility to permute a set of values.



TraceVariable	A class containing a name/value/location tuple.
TraceVariableList	A list of TraceVariables
TraceVariableStatement	A statement wrapping a TraceVariable.

Enum Summary	
Verdict	A test outcome: PASSED, FAILED or INDETERMINATE.
TraceReductionType	A reduction type, RANDOM or various SHAPE types

The traces sub-package includes classes to represent the definitions which can occur in a VDM-SL, VDM++ or VDM-RT "traces" section, and their subsequent expansion and execution.

The subclasses of TCTraceDefinition are uncontroversial and follow the structure of the trace grammar closely. These classes have the usual typeCheck methods which permit the trace specifications to be checked as part of the overall specification type check phase.

In order to evaluate traces, they must first be expanded into all the possible execution paths represented by the definition. For example, a trace that is of the form "a;(b|c);d" would expand to "a;b;d" and "a;c;d". Similarly, "a{1,3}" would expand to "a", "a;a" and "a;a;a". Repeats, sequences, alternations and permutations multiply together to generate large numbers of tests very quickly – this is the whole point of combinatorial test specification. The Traceliterator class and its subclasses represent the expanded traces, and are generated via the "getIterator" method on all INTraceDefinitions. The getNextTest and hasMoreTests methods of Traceliterators actually expand the tests, returning a CallSequence, which is a list of Statements (INCallStatements, INCallObjectStatements or INTraceVariableStatements).

All Traceliterators include a field called variables, which may contain an INTraceVariableList. These are populated by INTraceLetBeStBinding and INTraceLetDefBinding when a particular name/value pair is chosen in the expansion of a give test sequence. The getVariables method of Traceliterator returns a CallSequence which is populated with any INTraceVariableStatements for the expansion. When executed, these statements just add their named value to the local context, making it available to subsequent calls in the test.

A top level INNamedTraceDefinition (see below) is created for each parsed trace definition, including a method to obtain an iterator to extract all the tests from the definition tree it contains.

The execution of a trace is coordinated from a method called runtrace on the Interpreter object (see below). This first gets an iterator from the INNamedTraceDefinition. Then for each CallSequence returned, it initializes the interpreter, and calls its runOneTrace method, passing the CallSequence. The runOneTrace method executes the CallSequence passed and returns a list of java.lang.Object, being either the return Values from the test steps or error messages, where the last item will always be an instance of a Verdict object, indicating the test outcome.

The runtrace and runOneTrace methods have a boolean debug argument. If true, this allows errors in trace execution to trap into the debugger, otherwise such errors are caught and returned in the list of results without interrupting the execution of the trace.

Tests which have a FAILED verdict are "stemmed" by TraceFilter such that remaining tests from the iterator which have the same stem are marked as "filtered" by this test – ie. there is no point in running them because their initial sequence of calls will fail at the same point, for the same reason. Before each test is executed, its filtered flag is tested, and such tests are not executed. Note that the stem check includes the value of INTraceVariableStatements in the test. This means that tests which are superficially the same, but which have different variable values will not match.

The TraceFilter class is also used to cut down a very large collection of tests before execution. The constructor is passed three parameters: the proportion of the tests to keep (a float value between 0 and 1), a TraceReductionType which indicates the type of reduction to perform, and a random seed value. The simplest reduction type is RANDOM, which seeds a PRNG and then randomly removes tests until the required number remain. The weakness of this type of reduction is that it does not guarantee to preserve all the possible "shapes" of operation call sequences in the test collection, a shape being a sequence of named operation calls (regardless of the arguments passed). The



SHAPES reduction type guarantees to keep at least one test of each test shape. There are three subtypes, SHAPES_NOVARS which ignores TraceVariableStatements when determining shapes, SHAPES_VARNAMES which takes variable names into account but not their values, and SHAPES_VARVALUES which considers variables and their values.

2.5.5.1. Comments

There is a minor quibble in the parsing of trace sections, in that the grammar prohibits the use of semi-colons between trace definitions (while permitting them to separate parts of a trace definition). VDMJ permits these separate semi-colons; the Overture parser currently does not.

The intention of the TraceVariable was to hold information about the location of a particular value from a set of (possibly) anonymous values, such as "let x in set {new A(1), new A(2), ...} in ..." Here, x will take one of the values from the set in each expansion, but it will always be called "x" and it is hard to distinguish cases when a given test fails. Unfortunately, this is very hard to achieve without giving (pure) Values a location. Currently, the location stored with TraceVariables is the location of the name of the variable. The name/value is included in the iterator (via the INTraceVariableStatement), so debuggers can look at the raw value, which may be of some help.

2.5.6. definitions

Class Summary	
(AST TC IN PO)Definition	The abstract parent of all definitions.
AccessSpecifier	A class to represent a [static] public/private/protected specifier.
AssignmentDefinition	A class to represent assignable variable definitions.
ClassDefinition	A class to represent a VDM++ or VDM-RT class definition.
SystemDefinition	A class to represent a VDM-RT system class definition.
CPUClassDefinition	A class to represent a VDM-RT CPU definition.
BUSClassDefinition	A class to represent a VDM-RT BUS definition.
ClassInvariantDefinition	A class to hold a class invariant definition.
EqualsDefinition	A class to hold an equals definition.
ExplicitFunctionDefinition	A class to hold an explicit function definition.
ExplicitOperationDefinition	A class to hold an explicit operation definition.
ExternalDefinition	A class to hold an external state definition.
ImplicitFunctionDefinition	A class to hold an implicit function definition.
ImplicitOperationDefinition	A class to hold an explicit operation definition.
ImportedDefinition	A class to hold an imported definition.
RenamedDefinition	A class to hold a renamed import definition.
InheritedDefinition	A class to hold an inherited definition in VDM++.
InstanceVariableDefinition	A class to hold and instance variable definition.
LocalDefinition	A class to hold a local variable definition.
MultiBindListDefinition	A class to hold a multiple bind list definition.
MutexSyncDefinition	A class to hold a mutex synchronization definition.
PerSyncDefinition	A class to hold a permission synchronization definition.
StateDefinition	A class to hold a module's state definition.
ThreadDefinition	A class to hold a thread definition.
TypeDefinition	A class to hold a type definition.



UntypedDefinition	A class to hold a definition of, as yet, an unknown type.
ValueDefinition	A class to hold a value definition.
NamedTraceDefinition	A class to hold a named trace definition.
ClassList	A class for holding a list of ClassDefinitions.
DefinitionList	A class to hold a list of Definitions.
DefinitionSet	A class to hold a set of Definitions with unique names.

The definitions sub-package contains classes representing the definitions in a specification. They are all sub-classes of an abstract Definition class.

All definitions have a few things in common (fields of the abstract class). They belong to a Pass, which guides the type checking for TCDefinitions; they have a location; they have a name, though this may be null if they contain sub-definitions; and they have a NameScope to define what sort of name(s) they define, which complements the name scope used in type checking that searches for names of certain types.

TCDefinitions define a typeResolve method which is used very early on to resolve the TCUnresolvedTypes that may have come through from the syntax analysis. For example, a TCExplicitFunctionDefinition would typeResolve the TCType of the function, and if there were pre or postconditions, these expressions would be typeResolved, and any parameter patterns would be typeResolved.

TCDefinitions also define a typeCheck method, which is similar to the ones defined for TCExpression and TCStatement, except that there is no TCType to return. For example, the TCExplicitFunctionDefinition performs the following tasks in its typeCheck method:

- If there are any polymorphic type parameters for this function, check that the overall function type does not reference any type parameters except those named type parameters.
- For each type parameter, create a LocalDefinition of a ParameterType and add this to a local Environment.
- Check that the parameter patterns match the overall TCType's parameters, and iterate through curried sets of parameters, using the return value from the overall TCType (and its return value and so on for subsequent sets of parameters). Remember the expected result.
- Extend the local Environment with definitions for all the variables of all the patterns from all of the curried parameter sets.
- Type check the definitions this produced in the base environment (this will just do type resolution, if necessary).
- Label the local Environment as static (VDM++) if the definition's access specifier is static.
- If we are in VDM++ or VDM-RT and the function is not static, add a "self" definition to the local Environment.
- If there is a precondition TCExpression, type check the definition for it.
- If there is a post condition TCExpression, type check the definition for that too.
- Type check the body TCExpression of the function, remembering the actual type returned.
- If the actual return type is not assignable to the expected return type, raise an error.
- If the VDM++/VDM-RT accessibility of the expected return type is narrower than that of the definition itself, raise an error (eg. a public function cannot have a private return type).
- If the function is recursive and does not define a "measure" function, raise a warning, else if there is a measure defined, check that it exists and has the correct type.

- Check that the parameter variables have been referenced in the local Environment, else raise an unused parameter warning. (This is suppressed for pre and post conditions, which are permitted to not necessarily use their implicit parameters).
- Return.

This illustrates the principles that are used by all definitions' typeCheck methods.

Some definition types are only used to "wrap" others. For example, during module imports and exports, a definition may be imported and/or renamed. These methods just delegate their calls to the referenced definition that they wrap.

As with Patterns, definitions can yield their contained definitions or a list of names or name/value pairs that they define. This is the purpose of the getDefinitions, getVariableNames (in TCDefinitions) and getNameValuePairs methods (in INDefinitions). Simple local definitions only define one variable, but many definition types include a Pattern specifier that may define many variables.

The findName method is implemented by all TCDefinitions to return whether they define a name being sought by type checking. As above, for simple definitions, this just compares their name and scope with that being searched for, but for definitions that include patterns, all the names generated by the pattern must be considered.

The findType method is implemented by those TCDefinitions that define a type (TCTypeDefinition, TCStateDefinition and TCClassDefinition).

INDefinitions also define findExpression and findStatement methods which recurse into their bodies in search of an expression or statement that starts on the given line.

The ClassDefinition class is slightly different from the others in that its main job is to contain the definitions in a class, though INClassDefinition does have the job of setting up the static and instance environment for new objects when a "new Object()" statement is executed. TCClassDefinition is responsible for stitching together the class hierarchy and arranging for symbols to be inherited so that type checking may be performed. The hierarchy is built during the generation of implicit definitions.

Note that class static data (eg. instance variables that are declared static) is held inside the INClassDefinition at runtime (in the public/privateStaticValues fields), whereas object instance data is held inside an ObjectValue (produced by the makeInstance method of INClassDefinition), though references to the static data is included in the object's member list, so that the runtime can find them. See section 2.8 for more information about runtime variable access.

The SystemDefinition class is a subclass of ClassDefinition, and adds VDM-RT specific processing for system classes. The implicit definition generation (see section 2.11) in TCSysDefinition checks whether the definitions in the system class meet the VDM-RT restrictions. A systemInit method is called during system initialization in INSystemDefinition and creates the necessary CPU and BUS objects from the system definition. The newInstance method is overridden, since it is not legal to create an instance of a VDM-RT system class.

The CPUClassDefinition and BUSClassDefinition classes represent VDM-RT CPU and BUS classes respectively. These are also subclasses of ClassDefinition, and (in the IN classes) override the newInstance method to perform special processing. Both TC classes create their operations (ie. create TCExplicitOperationDefinitions for their definition list) by parsing a string literal representing the operations required. For example:

```
private static String defs =
    "operations " +
    "public CPU:(<FP>|<FCFS>) * real ==> CPU " +
    "    CPU(policy, speed) == is not yet specified; " +
    "public deploy: ? ==> () " +
    "    deploy(obj) == is not yet specified; " +
    "public deploy: ? * seq of char ==> () " +
    "    deploy(obj, name) == is not yet specified; " +
    "public setPriority: ? * nat ==> () " +
    "    setPriority(opname, priority) == is not yet specified;";
```



```
private static TCDefinitionList operationDefs() throws Exception
{
    LexTokenReader ltr = new LexTokenReader(defs, Dialect.VDM_PP);
    DefinitionReader dr = new DefinitionReader(ltr);
    dr.setCurrentModule("CPU");
    ASTDefinitionList ast = dr.readDefinitions();
    return ClassMapper.getInstance(TCNode.MAPPINGS)
        .checkInit().convert(ast);
}
```

This technique permits the rest of the code to use these operations as normal. The *is not yet specified* processing intercepts the operation calls for CPU and BUS, calling back to the INCPUClassDefinition and INBUSClassDefinition classes to perform the actual processing.

2.5.6.1. Comments

There is a great deal of commonality between some definitions, especially implicit and explicit functions and operations. It might be possible to simplify the code by creating abstract bases for these.

2.5.7. modules

Class Summary	
Export	The parent class of all export declarations.
Export***	A class for representing exports of a given type.
Import	The parent class of all import declarations.
Import***	A class for representing imports of a given type.
Module	A class holding all the details for one module.
ModuleList	A list of Modules.

The modules sub-package contains value classes that represent the modular structure of VDM-SL specifications, including their import and export declarations.

The only purpose of these classes is to represent the parsed module structures from the specification, and to generate/find the list of exported and imported definitions that extend the scope of what is visible from a single module. The typeCheck method of TCImport and TCExport is implemented in the various subclasses to check that (say) the export definition matches the actual definition being exported.

The INModuleList class is important because it contains the "initialize" method which is used to set the initial state of all modules when the interpreter is started.

2.5.8. visitors

Class Summary	
<Plugin><Kind>Visitor	The abstract parent class of all visitors for this kind.
<Plugin>Leaf<Kind>Visitor	An abstract searching visitor
<Plugin>***Visitor	A visitor class

The visitors sub-package contains the visitor framework and visitor instances for each analysis package and kind within that package (definitions, expressions, etc.)



There is always an abstract parent visitor called something like TCDefinitionVisitor or POTypeVisitor. These just contain methods called case<NodeType>, and have generic type arguments to supply a single argument and the return type. For example:

```
public abstract class TCDefinitionVisitor<R, S>
{
    abstract public R caseDefinition(TCDefinition node, S arg);

    public R caseAssignmentDefinition(TCAssignmentDefinition node, S arg)
    {
        return caseDefinition(node, arg);
    }
    ...
}
```

Note that the methods all call an abstract caseDefinition(node, arg) method by default. This is so that default processing for all definitions (in this example) can be added to one method. Concrete visitors then extend this class and override the cases that they want to process; other cases will default to the caseDefinition method, which has to be implemented and provides a default value.

One common subclass of each type of visitor is called a “Leaf” visitor (eg. TCMLeafDefinitionVisitor). These process a definition and (potentially) visit each of its sub-clauses, using specialised visitors for expressions, statements, patterns and so on that it contains. The result is a collection of values. This enables an entire tree of a definition (in this example) to be processed, where the work is really performed at the leaves of the tree (hence the name); the abstract Leaf visitor provides the common searching framework, collecting values from its branches. For example:

```
abstract class TCMLeafDefinitionVisitor<E, C extends Collection<E>, S>
    extends TCDefinitionVisitor<C, S>
{
    protected TCVisitorSet<E, C, S> visitorSet = new TCVisitorSet<E, C, S>()
    {
        @Override
        protected void setVisitors()
        {
            // Default just includes this
            definitionVisitor = TCMLeafDefinitionVisitor.this;
        }

        @Override
        protected C newCollection()
        {
            return TCMLeafDefinitionVisitor.this.newCollection();
        }
    };

    abstract protected C newCollection();

    @Override
    public C caseAssignmentDefinition(TCAssignmentDefinition node, S arg)
    {
        C all = visitorSet.applyTypeVisitor(node.getType(), arg);
        all.addAll(visitorSet.applyExpressionVisitor(node.expression, arg));
        return all;
    }
    ...
}
```

The generic parameters for the leaf visitor define the Element of the collection to be returned from each method, the type of the Collection itself (a set or list etc) and the type of the argument to be passed. An abstract newCollection method has to be defined to create a new empty collection, and a VisitorSet is used to define the set of related visitors that will be called as the process searches through the expressions, statements, types, patterns etc that comprise the tree.



An abstract leaf visitor defines a default visitor set that just contains itself. But usually, concrete leaf visitors override the default visitor set, and the methods to process node leaves. The same visitor set is made available to all visitors in the set via their constructors so that they can pass calls between each other as necessary. For example:

```
public class TCGetFreeVariableDefinitionVisitor
    extends TCLeafDefinitionVisitor<TCNameToken, TCNameSet, Environment>
{
    public TCFreeVariableDefinitionVisitor()
    {
        visitorSet = new TCVisitorSet<TCNameToken, TCNameSet, Environment>()
        {
            @Override
            protected void setVisitors() // Called from TCVisitorSet ctor
            {
                definitionVisitor = TCFreeVariableDefinitionVisitor.this;
                expressionVisitor = new TCFreeVariableExpressionVisitor(this);
                statementVisitor = new TCFreeVariableStatementVisitor(this);
                patternVisitor = new TCFreeVariablePatternVisitor(this);
                typeVisitor = new TCFreeVariableTypeVisitor(this);
                bindVisitor = new TCFreeVariableBindVisitor(this);
                multiBindVisitor = new TCFreeVariableMultipleBindVisitor(this);
            }

            @Override
            protected TCNameSet newCollection()
            {
                return TCFreeVariableDefinitionVisitor.this.newCollection();
            }
        };
    }
    ...
}
```

VisitorSets can theoretically include just the leaf visitors that they need. But in practice, there are two sets of the possible visitors that make sense: all of them, and {Expression, Pattern, Bind, MultiBind, Type}. A VisitorSet instance can be created and passed to the members of a set, but it is more usual to have a “lead” constructor in the set that creates one and passes itself to the others, as shown above.

The following table row shows which visitor types use (green) which others in a set. Definition and Statement leaf visitors use everything; Type leaf visitors don't use anything.



	Definition	Statement	Expression	Pattern	Bind	Multibind	Type
Definition							
Statement							
Expression							
Pattern							
Bind							
MultiBind							
Type							

2.5.9. Top Level

Class Summary	
XXNode	A MappedObject abstract root for all analysis classes.
XXVisitorSet	A collection of visitors for this analysis type.
XXMappedList/Set/Map	Generic types which support the class mapping of the analysis.

The top levels of each analysis package have similar classes, specialised for the particular analysis.

The XXNode class extends MappableObject is extended by every analysis node below it. This allows the analysis classes to take part in class mapping. See section 1.2. Similarly, an XXMappedList, Set or Map is useful for mapping generic Lists, Sets and Maps of objects for the analysis.

The XXVisitorSet is described in section 2.5.8.

2.6. vdmj.typechecker

Class Summary	
TypeChecker	The abstract root of all type checker classes.
ClassTypeChecker	A class to coordinate all class type checking processing.
ModuleTypeChecker	A class to coordinate all module type checking processing.
Environment	The parent class of all type checking environments.
FlatEnvironment	Define the type checking environment for a list of local definitions.
FlatCheckedEnvironment	Define the type checking environment for a list of local definitions, including a check for duplicates and name hiding.
ModuleEnvironment	Define the type checking environment for a modular specification.
PrivateClassEnvironment	Define the type checking environment for a class as observed from inside.
PublicClassEnvironment	Define the type checking environment for a set of classes, as observed from the outside.



TypeComparator	A class for static type checking comparisons.
TCRecursiveCycles	A map to analyse recursive loops for the type checker.

Enum Summary	
NameScope	An enum to represent name scoping.
Pass	An enum to indicate which type checking pass a definition belongs to.

The `vdmj.typechecker` package includes classes which coordinate the type checking of VDM-SL, VDM-RT and VDM++ specifications. Most of the actual type checking is performed by the `TCDefinition`, `TCExpression` and `TCStatement` subclasses that collectively describe the specification (above), but the typechecker package defines the supporting classes to invoke the type checking methods of the other objects, and to represent the static environment in which type checking is performed.

Type checking is different for the three dialects, though the checking of the basic statements and expressions is very similar, and VDM-RT is really an extension of VDM++. Therefore there is one common abstract `TypeChecker` class with two subclasses: `ModuleTypeChecker` and `ClassTypeChecker`. The subclasses are constructed with a list of modules or classes – which is the overall result of a successful syntax analysis, converted into `TCNodes` – and they implement a single abstract method from their parent, called `typeCheck`. The method takes no arguments and returns no result. Any errors or warnings raised during type checking are recorded by the `TCNode` sub-class (see `VDMMMessage`).

The sequence of events is slightly different for the type check of modules and classes, but they follow the same principles. For modules, the sequence is:

- Check for duplicate module names in the list passed
- For each module, generate its definitions' implicit definitions (like pre and post functions)
- For each module, check the export definitions exist and are of the declared type, and make a list of exported definitions for the module.
- For each module, go through the import definitions and resolve against the exports.
- Create a list of all definitions from all modules (including their imports), create an `Environment` that contains them all, and attempt to perform type resolution on them – ie. find the type definition for every named type.
- In the pass order: [types, values, definitions], for each module, create a `ModuleEnvironment` representing the visible definitions, and type check the definitions of the given pass. This calls the `typeCheck` method on the definitions, which calls the similar method on the definitions subparts, if any.
- Call the `TCRecursiveCycles` `typeCheck` method to analyse mutually recursive loops.
- Report any discrepancies between the final checked types of the modules' definitions and their explicit imported types.
- Any definition names that have not been referenced or exported produce "unused" warnings.
- Perform a cyclic dependency check for all modules.
- Generate the transitive closure of operations' updates and check "ext" clauses.

There are a couple of important points to note:

Firstly, the syntax analysis does not understand anything about the relationship of a type name in a declaration to its type definition. All type names come through from the syntax phase as `TCUnresolvedTypes`, which simply have a name. So a very early phase of the type checking must

find the corresponding type definition, in order to understand the structure of the type and what is/is not a legal type manipulation. This is done on a global basis, even though not all types are in scope for a module (all type names are fully qualified with a module name, so there is no ambiguity). A subsequent pass, which uses just those definitions that are visible, will subsequently spot any scope problems.

Secondly, “environments” are used to support the type checking. An environment (a subclass of the abstract Environment class) is essentially a list of names and corresponding definitions that are in scope at any point. The concrete subclasses allow the different scope rules to be followed, so for example a module’s type checking will start with a ModuleEnvironment that references a single module definition, and understands the rule about resolving names from its imported definitions and its own definitions. Different environments are then chained together, so when the module environment is passed to (say) a function definition, the type checking creates a new environment with local definitions for the names that are generated by the parameter patterns. Since the parameter names are in scope for the body of the function, the chain of two environments is passed to the type checking of the body expression. That may in turn involve “let” expressions that define further local variables that are chained onto the environment before their bodies are type checked, and so on. As the chain unwinds (as typeCheck methods return), each environment in the chain is checked to see whether all of its definitions were referenced; any that were not referenced generate “unused” warnings.

The two most important methods on an Environment subclass are findName and findType, which lookup definitions by name (using the same methods on the TCDefinition classes they reference). There is also a name scope parameter passed to findName to indicate what sorts of names are sought – for example, functions “see” value and parameter names, operations see these names and state variables, and the post conditions of operations see names, state and “old” names. The name scope (mask) is passed around the tree of typeCheck invocations as the names that are visible in a given context is generally only known by the caller – eg. an expression does not know that it is part of an operation, so it must be told from the outside that state variables are in scope.

Very similar principles are followed by the ClassTypeChecker, which performs the following actions:

- Make sure there are no duplicate class definitions.
- For all classes and their definitions, generate the implicit definitions. This includes the construction of the class type hierarchy and the implicit local names for access to inherited definitions. VDM-RT specifications limit what can be done in system classes here.
- Create a PubliClassEnvironment that can see all public class definitions.
- For each class, chain a PrivateClassEnvironment to the public environment, and perform type resolution on the definitions in the class.
- For each class, check for overloading and overriding of its definitions.
- In the pass order: [types, values, definitions], for each class, create a PrivateClassEnvironment, and type check the definitions of the given pass.
- Check for any definition names that have not been referenced and produce “unused” warnings.
- Perform a cyclic dependency check for all classes.
- Generate the transitive closure of operations’ updates and check “ext” clauses.

The use of public and private class environments to control the resolution of names is exactly analogous to the module case, though the class versions use the static/public/protected/private definition modifiers to decide on visibility.

The TypeComparator is used at the heart of type checking. The “compatible” method is used to decide whether two TCTypes are assignment compatible (with “possible” semantics). This involves finding the “underlying” type (eg. removing the names of types) and making flexible recursive comparisons where unions are involved. It also has to have recursive defence, since type structures may reference themselves. Two optional types are always considered compatible (as they could both



be nil). Compound types that involve more than one subtype (sets, sequences, maps, functions and operations, records and classes) are unpicked and their subtypes recursively compared. Finally, a `Type.equals` comparison is made between simple types.

The `TypeComparator` is also involved in proof obligation generation for subtype testing (see below. This is effectively a "definite semantics" check, since PO generation is concerned with covering the gaps that "possible semantics" leaves). The `isSubType` method takes two types and returns a boolean indicating whether the first is a subtype of the second – for example, a `nat1` is a subtype of a `real` (all `nat1` values are real values), but not the other way round. With union types, a simple type is a subtype of a union if it is a subtype of any of the union members; a union type is a subtype of another union if every member of the first union is a subtype of the second union.

The `TCRecursiveCycles` class handles the analysis of mutually recursive loops in the type checker. This is a singleton, and is initially populated with call data from the type analysis of the specification – the `addCaller` method adds a call of a function or an operation from a parent definition. When all calls have been added, the `typeCheck` method on the singleton is invoked from the main `TypeChecker` process. This looks at the call graph, trying to find cycles (up to a fixed depth, current 8 calls). When all cycles have been found, they are checked for the absence of measures and for the consistency of measure types along each cycle.

The recursive cycles that apply to each caller are passed back to that calling node (for example a `TCApplExpression` or a `TCCallStatement`) in a `TCDefinitionList` field called `recursiveCycles`. When the TC tree is subsequently mapped to a PO tree, these loops are available for the generation of related proof obligations by the PO classes.

2.6.1. Comments

I had a lot of trouble getting the order of the initialisation and type checking right to be able to deal with all the specifications in the test suite. I'm not 100% sure that there aren't still obscure orderings of declarations that will defeat it.

The `TypeComparator` is currently a static class, which means that its methods need to be synchronized to work with VDM++/VDM-RT threads (it has state for recursion defence). This isn't really necessary, but there are quite a few places where the code would have to create a new `TypeComparator` instance if this is changed.

Type checking error messages are produced by static methods on the `TypeChecker` class, and the error count is held statically there too. This is because it is otherwise difficult to find the type checking object instance deep in a `typeCheck` call chain. This is in contrast to the syntax analysers, where a `Reader` keeps track of the errors for itself (plus any from the `Readers` it creates).

2.7. vdmj.pog

Class Summary	
<code>ProofObligation</code>	The abstract root of all proof obligations.
<code>***Obligation</code>	A particular type of proof obligation.
<code>ProofObligationList</code>	A list of proof obligations.
<code>POContext</code>	The abstract root of all obligation contexts.
<code>PO***Context</code>	A particular type of obligation context.
<code>POContextStack</code>	A stack of obligation contexts.
<code>POGState</code>	State information for operation PO processing

Enum Summary	
<code>POType</code>	An enumeration of the various proof obligation types.



The `vdmj.pog` package defines classes that support the proof obligation generator. Most of the actual obligation generation is performed by `PODefinitions`, `POExpressions` and `POStatements`, which include a `getProofObligations` method that returns a `ProofObligationList`, but the coordination classes are in the `vdmj.pog` package. A typical proof obligation contains a stack of nested "contexts" in which a particular obligation must be determined, plus a specific obligation to check. Therefore there are two important class hierarchies in the `pog` package: the `POContext` hierarchy, and the `ProofObligation` hierarchy.

Proof obligations can come from any part of a specification, but there are important differences between those from a function body and those from an operation body, so we cover these separately below.

2.7.1. Function Obligations

Function obligations come from sub-expressions within a function body. For example, a simple function like:

```
f: int * map int to int -> int
f(i, m) == if i < 10 then m(i) + 1 else m(i) - 1;
```

would generate two proof obligations, requiring that the two `m(i)` map applications are correct in the "then" and "else" branches, respectively:

```
Proof Obligation 1: (Unproved)
f: map apply obligation in 'DEFAULT' (test.vdm) at line 3:31
(forall i:int, m:map int to int &
  ((i < 10) =>
    i in set dom m))

Proof Obligation 2: (Unproved)
f: map apply obligation in 'DEFAULT' (test.vdm) at line 3:45
(forall i:int, m:map int to int &
  (not (i < 10) =>
    i in set dom m))
```

Notice that both obligations contain an outermost "forall" that represents the possible function parameter values, and that the "if" test value is determined to be true or false in order to check the map application in the two branches. These two form the "context" of the proof obligation; the last line in both POs is the actual obligation, which tests that the argument is within the domain of the map.

The outer forall context is represented by a `POFunctionDefinitionContext` object, and the if/else contexts are represented by `POImpliesContext` and `PONotImpliesContext` objects, respectively. The proof obligation itself is a `MapApplyObligation`. The two contexts form a stack for each obligation, and these are held by a `POContextStack`, which extends `Stack<POContext>`.

To generate proof obligations for an entire specification, an empty `POContextStack` is created and the `getProofObligations` method of each `PODefinition` is called, passing the stack. Depending on the definition type, the implementation may add to the context (eg. a function definition would push a `POFunctionDefinitionContext`) before calling `getProofObligations` for their inner `POExpression(s)` or `POStatement(s)`. In the example above, the `getProofObligation` method of the `POIfExpression` that comprises the function body would be called. That in turn would push an `POImpliesContext` before generating obligations in its "then" sub-expression, popping the stack, and pushing a `PONotImpliesContext` before generating obligations for everything in the else-if list, and final else. Lastly it would pop the stack back to the state it found it in before returning a list of all the generated POs.

```
public ProofObligationList getProofObligations(POContextStack ctxt)
{
```

```
ProofObligationList obligations = ifExp.getProofObligations(ctxt);

ctxt.push(new POImpliesContext(ifExp));
obligations.addAll(thenExp.getProofObligations(ctxt));
ctxt.pop();

ctxt.push(new PONotImpliesContext(ifExp));           // not (ifExp) =>

for (ElseIfExpression exp: elseList)
{
    obligations.addAll(exp.getProofObligations(ctxt));
    ctxt.push(new PONotImpliesContext(exp.elseIfExp));
}

obligations.addAll(elseExp.getProofObligations(ctxt));

for (int i=0; i<elseList.size(); i++)
{
    ctxt.pop();
}

ctxt.pop();

return obligations;
}
```

Notice that the POIfExpression does not actually generate any proof obligations itself; it only sets up context so that obligations generated from its sub-expressions will be correct. The POApplyExpression actually generates the proof obligations in this example:

```
public ProofObligationList getProofObligations(POContextStack ctxt)
{
    ProofObligationList obligations = new ProofObligationList();

    if (type.isMap())
    {
        MapType m = type.getMap();
        obligations.addAll(MapApplyObligation.getAllPOs(
            root, args.get(0), ctxt));

        Type atype = argtypes.get(0);

        if (!TypeComparator.isSubType(atype, m.from))
        {
            obligations.addAll(SubTypeObligation.getAllPOs(
                args.get(0), m.from, atype, ctxt));
        }
    }
    ...
}
```

Here, the apply expression is first tested for whether it is a map application (it could be a function or operation application), and if so, a new MapApplyObligation is created, which is passed the context. Similarly, if the apply argument type is not a subtype of the domain of the map, a SubTypeObligation is also generated. The “getAllPOs” wrapper is used in operations to generate multiple POs (see below).

The constructor of the ProofObligations generated use the context passed to generate the "source" of the obligation (ie. the string form of its expression):

```
public MapApplyObligation(
    POExpression root, POExpression arg, POContextStack ctxt)
{
    super(root.location, POTYPE.MAP_APPLY, ctxt);
}
```

```
        source = ctxt.getSource(arg + " in set dom " + root);
    }
```

The `getSource` method on the context stack will generate a string composed of the contexts in the stack, plus the string passed in for the obligation. It is also responsible for the indentation and bracketing of the expressions. All `ProofObligation` subclasses are similar to the example above, though the more complex ones take a lot of effort to generate the string of the obligation. The most complex obligation is `SubTypeObligation`, which has a recursive private method to generate all the subtype tests that are required for the "structure" of the type being considered.

2.7.2. Operation Obligations

The process of proof obligation generation is similar for operation definitions which include statements rather than expressions. The outermost context in this case is a `POOperationDefinitionContext`. The important difference with the creation of operation POs is that, unlike function expressions, there is not always a unique path through the operation to get to the obligation point, and furthermore the environment includes state data, which is effectively another parameter to the operation.

For example, consider the following simple operation that uses state data:

```
state S of
    count : nat
end

...

op: nat ==> real
op(a) ==
(
    dcl x:nat := a + count;
    x := x + 1;
    return 1/x    -- Line 11
);
```

This generates the following obligation:

```
Proof Obligation 1: (Unproved)
op: non-zero obligation in 'DEFAULT' (test.vdm) at line 11:17
(forall a:nat, mk_S(count):S &
  (let x : nat = (a + count) in
    (let x : nat = (x + 1) in
      x <> 0)))
```

Comparing this to a function obligation, there are some important differences. Firstly, the "S" state vector is included in the outermost context, giving a "count" field value within the PO. Note that the PO is still a simple expression, not a statement, so the "dcl" statement adds a local "let" within the obligation, giving the PO a value for "x" for obligations in paths that include the dcl. Similarly, the assignment to "x" is also represented as a further "let" expression that *hides* the previous x value. Lastly, the obligation at the end is the same as the functional case, checking that x is not causing a divide by zero.

The example above has a single path to reach the obligation point (the 1/x). But this is not always the case with operations (unlike functions). Consider this updated operation:

```
op: nat ==> real
op(a) ==
(
    dcl x:nat := a + count;

    if a < 10
    then x := x + 1
```

```

        else x := x * 2;

        return 1/x    -- Line 16
    );

```

Proof Obligation 1: (Unproved)
op: non-zero obligation in 'DEFAULT' (test.vdm) at line 16:13
(forall a:nat, mk_S(count):S &
 (let x : nat = (a + count) in
 ((a < 10) =>
 (let x : nat = (x + 1) in
 x <> 0))))

Proof Obligation 2: (Unproved)
op: non-zero obligation in 'DEFAULT' (test.vdm) at line 16:13
(forall a:nat, mk_S(count):S &
 (let x : nat = (a + count) in
 (not (a < 10) =>
 (let x : nat = (x * 2) in
 x <> 0))))

Now we see that we do not know which route the operation took to get to the obligation point – the “then” or “else” clauses are different. Therefore *two obligations* are generated for the same base obligation, with each path represented in the POContextStack leading to it. In general, if there are N paths to get to an obligation point, then there will be N obligations produced.

To implement this state tracking, POStatements and POExpressions have an extra POGState parameter that they use to track local dcl variables. When the flow of control forks, for example into the then/else branches of a statement, two sub-POContextStacks created by the “then” and “else” branches and saved in a POAltContext, using the “copyInto” or “popInto” methods of the stack. Then a method called getAlternatives() produces a list of POContextStacks (ie. flattening any POAltContexts), representing each alternative path. Obligations then have a “getAllPOs” method which generates POs for each alternative:

```

public static List<ProofObligation> getAllPOs(
    LexLocation location, POExpression right, POContextStack ctxt)
{
    Vector<ProofObligation> results = new Vector<ProofObligation>();

    for (POContextStack choice: ctxt.getAlternatives())
    {
        results.add(new NonZeroObligation(location, right, choice));
    }

    return results;
}

```

Path expansion can become very large for long operations with many consecutive path choices. To avoid using too much memory, the expansion is limited by a property vdmj.pog.max_alt_paths. This is the maximum per operation, and defaults to 200. Beyond this, POG omits some paths and therefore POs on those paths. This rarely happens, but if it does, the “pog” command indicates there are missing POs from the operations. The solution is usually to break a long operation into smaller parts.

Note that (unlike functions) operations may leave contexts on the stack to influence obligations in later statements. So these must be popped from the stack as alternative paths are processed, which is supported by the pushAt and popTo methods on POContextStack.

So the POIfStatement’s getProofObligations looks like this (simplified):

```

public ProofObligationList getProofObligations(
    POContextStack ctxt, POGState pogState)
{
    POAltContext altContext = new POAltContext();

```

```

ProofObligationList obligations =
    ifExp.getProofObligations(ctxt, pogState, env);

int base = ctxt.pushAt(new POImpliesContext(ifExp));
obligations.addAll(thenStmt.getProofObligations(
    ctxt, pogState, env));
ctxt.popInto(base, altContext.add());
ctxt.push(new PONotImpliesContext(ifExp));

for (POElseIfStatement stmt: elseList)
{
    ProofObligationList oblist =
        stmt.elseIfExp.getProofObligations(ctxt, pogState, env);

    int popto = ctxt.pushAt(new POImpliesContext(stmt.elseIfExp));
    oblist.addAll(stmt.thenStmt.getProofObligations(
        ctxt, pogState, env));
    ctxt.copyInto(base, altContext.add());
    ctxt.popTo(popto);
    obligations.addAll(oblist);
    ctxt.push(new PONotImpliesContext(stmt.elseIfExp));
}

if (elseStmt != null)
{
    int popto = ctxt.size();
    obligations.addAll(elseStmt.getProofObligations(
        ctxt, pogState, env));
    ctxt.copyInto(base, altContext.add());
    ctxt.popTo(popto);
}
else // eg. for an if with no else
{
    ctxt.copyInto(base, altContext.add());
}

ctxt.popTo(base);
ctxt.push(altContext);

return obligations;
}

```

Operation calls add context to the stack which specifies the possible value(s) of the state variables that should be affected, qualified by any pre/postconditions that apply. For example, a call to an operation called “op2” whose return value is assigned to “rv” would add context like this:

```

(let $oldState = mk_Sigma(sv) in
  (forall sv:nat, $op2:nat &
    pre_op2((a + 1), $oldState) and
    post_op2((a + 1), $op2, $oldState, mk_Sigma(sv)) => -- Call to op2
    (let sv : nat = $op2 in ...

```

The “rv” value will be any nat which passes the post_op2 function, under the current Sigma state of the system, and a possible new state that is selected by the forall. Note that the context therefore represents *all possible operation outcomes*.

Operations may be recursive, in which case it is helpful to be able to verify that the recursion terminates, in a similar way that *measures* do with functions. Strictly, unlike functions, operations are not required to terminate – for example, *while* loops may loop forever. But in most cases, recursive operations are expected to terminate. This can be verified with using the @OperationMeasure annotation, which can only be applied to operation definitions. For example:

```
-- @OperationMeasure(a)
fac: nat ==> nat
fac(a) ==
  if a = 1
  then return 1
  else return a * fac(a-1);
```

This is a simple operation that implements a recursive factorial evaluation. The measure expression “a” is decreasing for every recursive call to the operation, because of the “a – 1” passed to the inner call. Without the annotation, no POs are generated for the recursion. The annotation causes the following PO to be generated:

```
(forall a:nat &
  (let MEASURE_3$ : nat = a in
    (not (a = 1) =>
      (let a : nat = (a - 1) in
        MEASURE_3$ > a))))
```

The *MEASURE_3\$* variable is a ghost which is set to the initial value of the measure, at the start of the obligation (the *_3* suffix indicates the line number; a meaningful name for the measure can be given in the annotation). At the point of recursion, the measure expression is re-evaluated in the context of the variable updates that were performed on the path. This new measure value must be less than the original one to ensure termination.

The production of obligations from while-loops and for-loops requires the use of loop invariants. These are added to a specification via *@LoopInvariant* annotations. For example:

```
dcl ax : set of nat1 := {};

-- @LoopInvariant(ax = { g * 2 | g in set GHOST }, GHOST)
for all x in set {1,2,3} do
(
  ax := ax union {x * 2};
);
```

Here, a set value “ax” is initially empty, and the loop populates the set with the double of a set of values, {1, 2, 3}. The “GHOST” variable is declared using the second parameter and is in scope for *@LoopInvariant(s)* and holds “the set of values processed in the loop so far”. This produces several obligations, including:

```
Proof Obligation 2: (Unproved)
preservation for next for-loop
op: loop invariant obligation in 'DEFAULT' (test.vdm) at line 9:9
(let ax : set of nat1 = {} in
  (forall GHOST:set of (nat1), x:nat1, ax:set of (nat1) &
    ((GHOST psubset ax)
     and (x in set ({1, 2, 3} \ GHOST))
     and (ax = {(g * 2) | g in set GHOST}) =>
      (let GHOST$ : set of (nat1) = (GHOST union {x}) in
        (let ax : set of nat1 = (ax union {(x * 2)}) in
          (ax = {(g * 2) | g in set GHOST})))))))
```

The obligation checks that, if the loop invariant is met at the start of a loop iteration, then the action of extending the GHOST set and performing the body statements leaves the system in a state where the loop obligation is still met at the end of the loop. Inductively therefore, the loop invariant is preserved for the whole loop execution.

Note that the GHOST variable definition is added to a new NameScope called GHOST. This keeps the variable inaccessible to normal statements and expressions in the loop, but the invariants in nested loops do have access to the ghost variables of all surrounding loops, as well as their own ghost.



2.7.3. Comments

It is possible that there are other obligation types that should be added, especially in the case of VDM++/VDM-RT. Furthermore, for operations, we have to solve the problem of how to represent the whole object system state in the outermost context. This is complicated by inheritance and static data. Many POs for these dialects are marked as “Unchecked”.

Other areas in operation POG produce “Unchecked” obligations, such as those to do with exception handling.

ProofObligation subclasses generate the entire string of the obligation, including the context in which it is generated. This seemed better than keeping the context objects with the obligation, but it does mean that the context structure of the PO is lost in the flat string.

The generation of expression strings depends on the accuracy of the toString methods for Expressions. These have been tidied up, but unfortunately preserving the precedence of the original operators means that many expression strings end up being excessively bracketed.

2.8. vdmj.runtime

Class Summary	
Interpreter	An abstract VDM interpreter.
ModuleInterpreter	The VDM-SL module interpreter.
ClassInterpreter	The VDM++ and VDM-RT interpreter.
Context	A class to hold runtime name/value context information.
RootContext	An abstract context for the root of a function or operation call.
ObjectContext	A root context for object member invocations.
StateContext	A root context for non-object member invocations.
ClassContext	A root context for static member invocations.
Breakpoint	The concrete root of the breakpoint hierarchy.
Stoppoint	A breakpoint where execution must stop.
Tracepoint	A breakpoint where something is displayed, but execution continues.
Catchpoint	A breakpoint when a particular exception is thrown.
SourceFile	A class to hold a source file for source debug output.
ThreadState	A class to hold runtime information for a VDM thread.

Exception Summary	
ContextException	A fatal interpreter error, including a “stack” context to dump.
DebuggerException	An exception used to stop the interpreter cleanly from the debugger.
ExitException	An exception used to implement exit statements.
PatternMatchException	A non-fatal exception indicating a pattern match has failed.
ValueException	A non-fatal exception concerning an evaluation.

The vdmj.runtime package defines the abstract interpreter, and its subclasses to interpret VDM-SL, VDM++ and VDM-RT specifications. It includes classes to represent the runtime execution context, exceptions which can be generated at runtime, and classes for breakpoint handling.

At its simplest, an interpreter has to create a runtime environment that represents the globally visible name/values in a specification, then evaluate a function or operation indicated by the user, returning the result.



The runtime name/value pair environment is held by the Context class and its sub-classes. A Context extends a `HashMap<TCNameToken, Value>`, so it is capable of storing and retrieving named values. In the same way that Environment objects were chained together during type checking, Context objects can be chained together as the evaluation creates new named values, and those names later disappear from scope.

Contexts allow named values to be retrieved by searching the present context, and then subsequent chained contexts. Hence a context for global variables might be chained with one for a function's parameters, and a further one defining variables in a "let" expression. Then a lookup of a variable would search this chain in reverse order.

In the case of functions or operations calling other functions or operations, the name searching should not proceed down the context chain beyond the nearest function or operation point – ie. if `func1` calls `func2`, `func1`'s variables are not in scope in `func2`. But global variables are visible in this case. Therefore a sub-class of Context, called `RootContext` is used to represent points in the context chain where the search should "jump" down to the global level. In fact there are three sub-classes of `RootContext`, one of which, `ObjectContext`, is specialized to hold an object value for "self" (which is in scope at the start of object member calls), another, `ClassContext` refers to an `INClassDefinition` for static invocations, and the third, `StateContext`, is able to have a module's state data attached (the actual state that is visible in VDM-SL changes as the operation call stack jumps from module to module, so this must be held in the context chain somewhere).

To create the global environment, the interpreter asks the default module or the `INClassList` passed to create the name/values from their definitions. The `INClassList` initialize method will dump all class' public static values into one global public static Context object; while the `ModuleList` initialize method will cause each module to initialize itself. The `ClassInterpreter` will then take the global public static scope and add the private static scope of the default class before starting. The `ModuleInterpreter` just takes the initial context from the default module before starting.

The code for the two interpreters' execute methods are similar therefore. Here is `ClassInterpreter`'s:

```
@Override
public Value execute(String line) throws Exception
{
    TCExpression expr = parseExpression(line, getDefaultName());
    typeCheck(expr);

    INExpression inexpr = ClassMapper.getInstance(INNode.MAPPINGS)
        .convert(expr);

    return execute(inexpr);    // below
}

private Value execute(INExpression inexpr) throws Exception
{
    Context mainContext = new StateContext(
        defaultClass.name.getLocation(), "global static scope");

    mainContext.putAll(initialContext);
    mainContext.putAll(createdValues);
    mainContext.setThreadState(CPUValue.vCPU);
    clearBreakpointHits();

    MainThread main = new MainThread(inexpr, mainContext);
    main.start();
    scheduler.start(main);

    return main.getResult();    // Can throw ContextException
}
```

Note that the public method is given a string expression to evaluate, so first it parses and type checks the expression given. The private method contains the important bit: a new `StateContext` is created, the global public static content is added, any user-created values are added, the thread state is

initialized to reference the virtual CPU (for VDM-RT), the breakpoint hit counts are cleared, the initial thread is set in the base scheduler, and lastly the expression passed is evaluated in the context constructed by creating a MainThread (see the scheduler section below) and using the interpreter's scheduler instance to coordinate any further threads that the evaluation may create. The return value is finally extracted from the completed main thread.

When processing reaches a breakpoint, the evaluation of the current expression or statement should suspend, and enter a state where the interactive user can examine the state of the system.

This is fairly simple with VDM-SL because the specification cannot have threads, therefore pausing at a breakpoint must pause the entire system, and the user can easily examine anything they wish to. However, with VDM++ and VDM-RT the presence of threads make debugging more complex as the user may want to examine the state of other threads at a breakpoint. Therefore during a debug session, the ConsoleDebugReader class is used to interact with multiple threads and display the result on the single output console.

The problem of single threaded consoles is conveniently sidestepped by using a GUI debugger, or more precisely, using the DBGp protocol to debug a remote VDMJ process. The advantage is that the protocol creates a separate TCP connection to the GUI for every thread that is created. This then avoids the problem of several threads wanting control of "the terminal". A command line DBGp client was provided for *real men* who prefer such things. Called VDMJC, this switches the console between connected DBGp threads by user command.

All breakpoint handling in VDMJ uses a Breakpoint class, and its subclasses. Every INStatement and INExpression has a breakpoint member object, and the eval methods of INExpressions and INStatements call their breakpoint's check method:

```
public abstract class Expression implements Serializable
{
    /** The textual location of the expression. */
    public final LexLocation location;

    /** The expression's breakpoint, if any. */
    public Breakpoint breakpoint;
```

and in subclasses ...

```
@Override
public Value eval(Context ctxt)
{
    breakpoint.check(location, ctxt);
    ...
}
```

The constructor for INExpression and INStatement just set the breakpoint field to a default Breakpoint. This base class' check method will not do anything under normal circumstances, but when a breakpoint is created by the user, the object in the expression or statement concerned is *replaced* with a subclass object, called Stoppoint. The check method of a Stoppoint will stop, although it may evaluate a conditional expression first.

When a Stoppoint stops, it allows the user to interact with the system at that point, evaluate expressions, print the stack and so on. This is done by calling some sort of command interpreter to read the console, or by calling into the DBGp session code to communicate with the GUI. However this is done, the important point is that when the user asks the thread to resume, control returns into the check method, and then out into the eval method of the stopped expression or statement.

If the user has issued a "step" or "next" or "out" command, we want to stop at the next line (skipping calls with "next", or until the current function/operation returns with "out"). This means that the Breakpoint base classes encountered as the execution proceeds have to check this – the system does not set any new breakpoints when stepping as it does not know where the code will go next. Therefore the check code for base class Breakpoint has to check the debugging state – and do so as efficiently as possible as this is the default code that is always called.



A simplified version of the Stoppoint version of check is as follows:

```
public void check(LexLocation execl, Context ctxt)
{
    location.hit();
    hits++;

    ... execute any conditional check and set stop variable

    if (stop)
    {
        Thread current = Thread.currentThread();

        if (current instanceof SchedulableThread)
        {
            SchedulableThread th = (SchedulableThread)current;
            th.suspendOthers();
        }

        DebugLink.getInstance().breakpoint(ctxt, this);
    }
}
```

The hit counters are to do with conditional breakpoint hits, and code coverage. Note that, because the check method is called for every INExpression and INStatement that is executed, it is a good place to record code coverage (against the location).

Any conditional expression will have been parsed (ie. done once) when the Stoppoint was created, and a test simply evaluates any boolean expression using the Context passed. Note that if the thread is about to stop, it also suspends the other threads. This is so that their state can also be examined by the debugger. After that, the DebugLink class makes a connection back to the ConsoleDebugReader; this returns when the user continues.

The check method of the base class Breakpoint has to handle “step”, “next” and “out” rather than conditional breakpoints. Notice that step will stop whenever the location is not equal to the one in the state – ie. we are on a different file/line (but not a different position on the same line):

```
public void check(LexLocation execl, Context ctxt)
{
    location.hit();
    hits++;

    ThreadState state = ctxt.threadState;

    if (Settings.dialect != Dialect.VDM_SL)
    {
        state.reschedule();
    }

    if (state.stepline != null)
    {
        if (!execl.equals(state.stepline))
        {
            if ((stepping) ||
                (next && !isAboveNext(ctxt.getRoot())) ||
                (out && isOutOrBelow(ctxt)))
            {
                new Stoppoint().check(location, ctxt);
            }
        }
    }
}
```

As well as marking the code coverage and breakpoint hit counter, the base Breakpoint's check method also calls the reschedule method of ThreadState which will conditionally terminate a timeslice if sufficient statements or expressions have been evaluated (see SchedulableThread below). Normally, this method just returns.

The isAboveNext and isOutOrBelow methods look at the context stack to decide whether to stop based on whether we are in a deeper stack frame (skip for next) or a shallower one (stop for out).

Notice that to actually stop, we create a Stoppoint and call its check method (which stops unconditionally), so it behaves as though you had inserted a breakpoint at the current location.

There is another type of breakpoint called a Tracepoint. This is very simple as it does not interrupt the flow of control in its check method, but rather evaluates some trace expression and prints its value to the console (or sends it to the GUI).

The third type of breakpoint is called a Catchpoint. This is intended to catch exceptions thrown by "exit" statements in the code. They are maintained by the interpreter in the usual way, but they are only checked in the INExitStatement code. If the value being thrown by the exit statement matches any Catchpoints, or there are any Catchpoints with a "null" value (ie. catch anything), then control trips into the debugger in the normal way.

2.8.1. Comments

The initialization of the VDM++ and VDM-RT runtime system is quite complicated, and should probably be cleaned up.

Here are a few points of general interest:

- The runtime system is initialized from the ClassList or ModuleList classes' initialize method.
- VDM++ and VDM-RT have to initialize the statics for each class first. This is done in two steps via methods on the ClassDefinitions. The first, staticInit creates the functions, operations and types – these do not have initializers; the second staticValuesInit covers instance variables and values, which can make calls to other static methods in their initializers.
- Value initializers can make forward references to other values which are not yet initialized. This causes characteristic exceptions, which are caught (ignored) and produce a second pass of the staticValuesInit initialization – up to a limit (currently 3). This ought to be done by working out the reference graph for the initializers.
- The static data thus initialized is written to a Context passed in (which becomes the global static environment). Static name value pairs are also written to maps inside the INClassDefinition – static data resides "in" the INClassDefinition at runtime.

And regarding object creation:

- The construction of an object (an ObjectValue) creates the superclass objects first, then for inherited fields, it adds a "locally named" **reference** to the superclass value. This is so that they can be explicitly referred to with a local name like C`x at runtime.
- Lastly, local definitions are used to override anything inherited, and the set of members are passed, together with the superclass objects, to the ObjectValue constructor.
- If the class defines an operation with the same name as the class (a constructor), it is called.

At runtime, an object's "self" field values are accessed via an ObjectContext which refers to the self ObjectValue. The check method of the context first searches the local contents for the name, then uses the "get" method of the ObjectValue (and lastly uses globals, if the name is not in self). The ObjectValue's get method is told whether the name is explicit or implicit (ie. whether it is qualified with a class name and a backtick), and uses this to either create a local name and search each object in the contained hierarchy, or uses the explicit name to search the hierarchy. If a static function or



operation is called, there is no self, only static values. These are managed via a `ClassContext`, which refers to an `INClassDefinition` rather than an `ObjectValue`. The `INClassDefinition`'s "get" method is conceptually the same as the `ObjectValue`'s except it searches the static values of the class hierarchy.

2.9. vdmj.scheduler

Class Summary	
ResourceScheduler	The master resource scheduler.
SystemClock	The system clock, used by VDM++ and VDM-RT.
Resource	An abstract resource, to be scheduled.
BUSResource	A BUS resource.
CPUResource	A CPU resource.
SchedulingPolicy	An abstract scheduling policy.
FCFSPolicy	The First Come First Served scheduling policy.
FPPolicy	The Fixed Priority scheduling policy.
SchedulableThread	An abstract thread that can be scheduled by the resource scheduler.
MainThread	The main thread of an evaluation
InitThread	A thread to run initialization
ObjectThread	A thread created by an object start statement (VDM++)
AsyncThread	An asynchronous thread (VDM-RT)
BusThread	A thread handling a BUS (VDM-RT)
PeriodicThread	A periodic thread (VDM++ or VDM-RT)
CTMainThread	A main thread for running combinatorial tests
ControlQueue	An exclusive lock for a resource
Holder<T>	A pool for synchronized passing of data
Lock	A lock for handling guards
MessagePacket	An abstract message packet
MessageRequest	An operation request packet
MessageResponse	An operation reply packet.

Enum Summary	
Signal	An instruction to a suspended thread.
RunState	A SchedulableThread's current run state.

The vdmj.scheduler package defines classes that schedule the deterministic execution of multiple threads, and for VDM-RT, coordinate the movement of simulated time.

VDM-SL specifications are simple single threaded evaluations. Their execution will always produce the same result (unless they use a random input). However, VDM++ and VDM-RT specifications can have multiple threads of execution, and potentially the evaluation becomes non-deterministic. In order to prevent this, VDMJ uses a scheduler which coordinates the activity of all threads in the system and allows them to proceed, according to a policy, in a deterministic order. This guarantees repeatable evaluations even for highly threaded specifications.

VDMJ implements VDM threads using Java threads. The Java language does not define the operation of the underlying JVM thread scheduler. Instead, Java places various constraints on the ordering of events that must occur between threads, in the light of synchronization primitives that control access to shared resources. This means that although a given Java program will have a well defined partial ordering of some of its operations, the JVM thread scheduler has a great deal of freedom regarding how to execute threads that are not using synchronized access to variables or

methods. In particular, there is no way to control which thread gets control of which (real) CPU in the system, or for how long, though this can be influenced by setting a thread priority.

Building on this, VDMJ has to use Java synchronization primitives to enforce the semantics of the various VDM language features to control concurrency (permission guards and mutexes), and to control the order and duration of timeslices allocated to the different threads.

VDMJ scheduling is controlled on the basis of multiple "Resources" by a "Resource Scheduler". A resource is a separate limited resource in the system, such as a CPU or a BUS. These are separate in the sense that multiple CPUs or BUSses may exist, and limited in the sense that one CPU can only run one thread at a time, and one BUS can only be transmitting one message at a time. Therefore there is a queue of activity that should be scheduled for each resource – threads to run on a CPU, or messages to be sent via a BUS. The Resource Scheduler is responsible for scheduling access to the resources in the system.

The ResourceScheduler class implements the master resource scheduler. One interpreter (of any dialect) has a single ResourceScheduler instance. The ResourceScheduler controls a list of Resource objects, each of which is added to the list via a register method. The abstract Resource class has concrete subclasses called CPUResource and BUSResource. A VDM-SL or VDM++ system will have only one CPUResource (called a *virtual* CPU) and no BUSResources; a VDM-RT system will have as many CPUs and BUSses as are defined in the "system" class.

Every Resource has a scheduling policy, potentially different for each instance of the resource. A policy extends the abstract SchedulingPolicy class, and implements methods to reschedule (to reconsider what is best to run next) and subsequently to identify the SchedulableThread (below) that is best to run next and for how long it is to be allowed to run (its timeslice).

With VDM-RT, in the event that the swapped-in thread is trying to move system time, the Resource will identify this fact. The ResourceScheduler is responsible for waiting until all resources are in this state, and then finding the minimum time step that would satisfy at least one of the waiters.

The ResourceScheduler loops through its resources with the following method outline:

```
do
{
    long minstep = Long.MAX_VALUE;
    idle = true;

    for (Resource resource: resources)
    {
        if (resource.reschedule())
        {
            idle = false;
        }
        else
        {
            long d = resource.getMinimumTimestep();

            if (d < minstep)
            {
                minstep = d;
            }
        }
    }

    if (idle && minstep has been set)
    {
        SystemClock.advance(minstep);

        for (Resource resource: resources)
        {
            resource.advance();
        }
    }
}
```



```
        idle = false;
    }
}
while (!idle && main thread is not COMPLETE);
```

Each Resource's reschedule method is responsible for determining the best thread to run next, if any, and to run it for the policy-determined timeslice. If the reschedule actually did anything (ie. the resource is not idle), it returns true. If it is idle, it is possible that the resource has its swapped-in thread waiting to perform a time step (for VDM-RT). In that case, the `getMinimumTimestep` method returns that value, and the scheduler uses it to find the global minimum time step. Note that this minimum time step includes periodic threads which are not yet running, but which are in an ALARM state waiting to be scheduled at a particular time. Their "minimum step" is the time until they are due to be started; when their `CPUResource` is advanced to (or past) this time, they are made runnable.

If all Resources have been considered, and the system is completely idle, and the global minimum time step has actually been set (the Resources may be idle because they are waiting on guards or messages rather than moving time), then global system time can be moved by the global minimum step. This is done via a call to `SystemClock`, which manages global time, followed by a call to each resource's `advance` method, which delegates to the policy's `advance` method. This will result in all threads which were previously in the `TIMESTEP` state (blocked trying to move time) being moved to the `RUNNABLE` state (these are `RunState` enum values), and any which are now ready to run will have their transaction variables committed. The scheduler then notes that it is not idle, and goes back to reschedule all the resources. Each of the `CPUResources` that was involved in the time step will reschedule the same swapped-in thread – this is because time steps do not cause a timeslice to terminate, but rather the thread is allowed to run until its slice expires or it enters a waiting state.

The reschedule method for the `CPUResource` is the most complex of the two resource types. It performs the following outline:

```
if (swappedIn thread is in state TIMESTEP)
{
    return false;
}

if (policy.reschedule())
{
    SchedulableThread best = policy.getThread();

    if (swappedIn != best) // We swapped
    {
        if (swappedIn != null)
        {
            RTLogger.log("ThreadSwapOut...");
        }

        if (delayed)
        {
            RTLogger.log("DelayedThreadSwapIn...");
        }
        else
        {
            RTLogger.log("ThreadSwapIn...");
        }
    }

    swappedIn = best;
    swappedIn.runslice(policy.getTimeslice());

    switch (swappedIn.getRunState())
    {
        case COMPLETE:
            RTLogger.log("ThreadSwapOut...");
    }
}
```

```
        RTLogger.log("ThreadKill...");
        swappedIn = null;
        return true;

    case TIMESTEP:
        return false;

    default:
        return true;
}

return true;
}
else
{
    return false;
}
```

The first check is to see whether the CPU's swapped-in thread (if any) is performing a TIMESTEP. If this is the case, the resource is blocked (idle) until the master scheduler can coordinate a time step, so the method returns immediately with a false return (idle).

Next, it uses the policy object to determine the best thread to run next by calling its `reschedule` method. If there is none (the CPU is idle) it returns false. Otherwise the best thread is picked up from the policy, and if it is not the one currently running on the CPU, the old one is swapped out and the new one is swapped in (possibly with a delay). The policy understands that if the CPU is paused while coordinating a VDM-RT duration, the same thread will be the best thread to run when the time is advanced (ie. stopping at a duration does not swap out the thread).

The new swapped in thread value is then set, and it is allowed to run for the timeslice given by the policy. When the timeslice is over (or the execution ends) if the thread is complete the thread is swapped out and killed, and true is returned to the resource scheduler because this resource did something (it was not idle); if the thread has entered the TIMESTEP state, false is returned (it is idle); otherwise true is returned since there may be other threads ready to run (which is determined on the next call to `reschedule`).

There are two policies implemented in VDMJ: `FCFSPolicy` and `FPPolicy`. They both extend `SchedulingPolicy`. These are actually the classes that maintain the list of threads allocated to a Resource (every Resource has a policy). The only difference between them is in the way they decide on a timeslice. The FCFS policy returns a fixed timeslice (ie. the execution of a fixed number of operations or expressions is permitted), whereas the FP policy returns a variable number, actually being identical to the number passed to `setPriority` (in VDM-RT). If a thread has no priority set, it defaults to the FCFS timeslice. This means that both policies give every thread a chance to run, but the FP policy gives higher priority threads a longer time on the CPU when they do run.

Both policies can have a certain amount of "jitter" added to their calculation of timeslices. This is set via the VDMJ properties file (see section 2.19). There is no jitter by default, which produces perfectly deterministic evaluations every time. However, this can hide problems with VDM thread synchronization that just happen to work because of the scheduling. Adding jitter will cause the thread scheduling to become slightly non-deterministic and therefore may expose such problems.

Back in the resource scheduler, if every thread is idle (and we cannot perform a time step), then one of two things has happened: the specification has completed because the main thread is complete, or we are deadlocked. The deadlock state is detected if the main thread is not complete and yet one of the resources has an "active" thread – ie. something which would be expected to proceed eventually, in state TIMESTEP or WAITING. If the specification is deadlocked, all threads are sent a DEADLOCKED signal, which trips them into the debugger. We wait for the main thread to end (from the debugger perhaps) before cleaning up.

```
if (main is not COMPLETE)
{
    for (Resource resource: resources)
```



```
        {
            if (resource.hasActive())
            {
                print("DEADLOCK detected");
                SchedulableThread.signalAll(Signal.DEADLOCKED);
                // Wait for main to terminate...
                break;
            }
        }
    }

    SchedulableThread.signalAll(Signal.TERMINATE);
```

Note that if the main thread is COMPLETE, we don't care about the state of the other resources. At the end of the execution, all threads are sent a TERMINATE signal, which causes them to throw a ThreadDeath exception (which kills a Java thread silently).

All thread resources that are scheduled by the resource scheduler via the Resource subclasses extend SchedulableThread (which in turn extends java.lang.Thread). The SchedulableThread's run method (ie. the body of the thread) is as follows:

```
@Override
public void run()
{
    reschedule();
    body();
    setState(COMPLETE);
    resource.unregister(this);

    synchronized (allThreads)
    {
        allThreads.remove(this);
    }
}

abstract protected void body();
```

The reschedule call basically turns the threads from state CREATED to RUNNABLE. Note that only one thread is ever RUNNING, but many can be RUNNABLE. The body is an abstract method that subclasses must provide, and after running the body, the state is changed to COMPLETE and the thread is unregistered from its resource and removed from the (static) list of all known threads.

The main resource scheduler and all the sub-resource scheduling calls run in the main Java thread. So as we've seen in the CPUResource reschedule above, a SchedulableThread is given a timeslice to run by the main thread calling the SchedulableThread's runslice method. That is responsible for putting the thread in the RUNNING state, running it for the given slice or until it changes its own state to something other than RUNNING, and then signalling the scheduler that it is done by returning from runslice.

This inter-thread coordination can only be achieved with Java synchronization primitives. The runslice method in SchedulableThread is as follows:

```
public synchronized void runslice(long slice)
{
    // Run one time slice - called by Scheduler
    timeslice = slice;
    waitWhileState(RunState.RUNNING, RunState.RUNNING);
}
```

The waitWhileState method puts the thread into the state given by the first argument, and then waits



on the `SchedulableThread`'s lock while the thread is in the second argument state – ie. set this thread as running, and don't come back until it is no longer running. There is a similar method called `waitUntilState`:

```
private synchronized void waitWhileState(  
    RunState newstate, RunState until)  
{  
    setState(newstate);    // Call Java's notifyAll  
  
    while (state == until)  
    {  
        sleep();           // Call Java's wait  
    }  
}  
  
private synchronized void waitUntilState(  
    RunState newstate, RunState until)  
{  
    setState(newstate);    // Call Java's notifyAll  
  
    while (state != until)  
    {  
        sleep();           // Call Java's wait  
    }  
}
```

The act of changing the state does a `notifyAll` on the `SchedulableThread`'s lock, so anything else that is calling `sleep()` will wake up and check the new state. So any thread calling `waitUntilState` that is waiting for it to become `RUNNING` will continue, and the scheduler will sleep until something else changes the state to something other than `RUNNING`, such as the thread calling the waiting method:

```
public synchronized void waiting()  
{  
    // Enter a waiting state - called by thread  
    waitUntilState(WAITING, RUNNING);  
}
```

This would be called by the `SchedulableThread` code when it decides that it needs to suspend in the `WAITING` state, such as when it is waiting for a message on a `BUS`. Similarly `SchedulableThread`'s `reschedule` method is called when the timeslice runs out to keep the thread runnable, but yield control:

```
private synchronized void reschedule()  
{  
    // Yield control but remain runnable - called by thread  
    waitUntilState(RUNNABLE, RUNNING);  
}
```

Notice that all of these methods are synchronized on the `SchedulableThread`'s lock. This is so that they can receive the `notifyAll`, as well as because they depend on shared values changed by other threads.

The sleep method can be woken up by a "signal" as well as a `notifyAll` state change. Signals are used to trigger the same behaviour in all threads (typically), such as when a deadlock is reached or when the evaluation terminates. The signal is sent via the (Java) thread's `interrupt` method rather than `notifyAll`. This causes any blocked wait calls to throw an `InterruptedException`. So the sleep method is as follows:



```
private synchronized void sleep()
{
    while (true)
    {
        try
        {
            wait();      // For a state change notify
            return;
        }
        catch (InterruptedException e)
        {
            handleSignal(signal);
        }
    }
}

private synchronized void setSignal(Signal sig)
{
    signal = sig;
    interrupt();
}
```

Depending on the signal set, the `handleSignal` method may kill the thread, or trap into the debugger (at a breakpoint or a deadlock). After leaving the debugger (ie. after a "continue" or "step") the loop goes back to wait for a state change. In this way, the debugger cannot change the scheduled flow of control, though the behaviour as perceived by the debugging user can be rather "strange".

These primitives for controlling scheduling are built into more complex calls to achieve the effects we want. For example, when a VDM-RT thread wants to perform a "duration" time step, it has to suspend and wait for the system to send through an "advance" call when time has changed, then it needs to find out whether it got as much time as it needed (or whether a smaller requirement was satisfied for another thread), suspending again with a smaller request if not. The support for this is in the `duration` method of `SchedulableThread`:

```
public synchronized void duration(long pause)
{
    // Wait until pause has passed - called by thread

    if (!inOuterTimeStep)
    {
        setTimestep(pause);
        durationEnd = SystemClock.getWallTime() + pause;

        do
        {
            waitUntilState(TIMESTEP, RUNNING);
            setTimestep(durationEnd - SystemClock.getWallTime());
        }
        while (getTimestep() > 0);

        setTimestep(Long.MAX_VALUE); // Finished
    }
}
```

The calls to `setTimestep` just set a value which is subsequently retrieved by the scheduler via the `CPUResource` when it is asking for the minimum time step. (Note that this is not performed if an "outer time step" is in progress – such as when a `cycles` or `duration` statement is timing a block of statements). The system time is checked in a loop that waits in state `TIMESTEP` until the time required (possibly in several smaller steps, if other threads want smaller amounts of time). Note that a pause of zero will still cause the thread to do one time step.

The duration call is used by CycleStatements and DurationStatements, but the most frequent use is by the simple execution of statements and expressions, which call the SchedulableThread's step method:

```
public void step()
{
    if (dialect is VDM_RT)
    {
        if (!virtual)      // vCPUs don't take any time
        {
            duration(Properties.rt_duration_default);
        }
    }
    else
    {
        SystemClock.advance(Properties.rt_duration_default);
    }

    if (++steps >= timeslice && !inOuterTimeStep)
    {
        reschedule();
        steps = 0;
    }
}
```

Note that VDM-RT calls duration; VDM++ simply advances the clock as it does not coordinate time between threads. If the timeslice for the thread has expired and we are not inside a "cycles" block, reschedule yields control to the resource scheduler while keeping the thread RUNNABLE.

Most of the subclasses of SchedulableThread do simple things that involve evaluating an expression and catching any thrown exceptions. The expression to run is passed to the thread constructor. In the case of AsyncThreads, this is passed via a MessageRequest that has come via a BUS and the result is returned on the same BUS via a MessageResponse. The client of an async thread (assuming there is one – this is the inter-CPU case, rather than an operation marked as "async") waits for the result to be placed in a Holder<T> (here, a Holder<MessageResponse>). This is simply a data area of the <T> type given, plus a ControlQueue to wait on. A ControlQueue is something which a thread gains exclusive access to (via join/leave methods), with block/stim methods to wait for/signal an event. These are used to protect BUS resources and for a BUS reply to be signalled back to the client waiting. The block method of a ControlQueue calls the waiting method of its SchedulableThread. A Lock is a similar coordination device, but this is used during the evaluation of guards on synchronized operations. In this case, we want all threads trying to acquire the lock to be able to queue for it (and wait), but for all of them to wake up when the guard value has changed. This is very similar to a Java synchronization lock, except implemented in terms of the resource scheduler primitives.

A PeriodicThread is slightly more interesting than the others. It has to perform its operation periodically, but it also has to do so under strict time control, with various options for constraining the behaviour. This is enabled (for VDM-RT) by calling the SchedulableThread's *alarming* method. This just marks the thread as being in state ALARM and notes the (absolute) time at which the thread should be made RUNNABLE.

```
public void start()
{
    super.start();      // Register the new thread as normal

    // Here we put the thread into ALARM state (rather than RUNNABLE)
    // and set the time at which we want to be runnable to the expected
    // start, which may have an offset and jitter. VDM-RT only.

    if (VDM-RT)
    {

```



```
        long wakeUpTime = expected;

        if (first execution)
        {
            if (offset > 0 || jitter > 0)
            {
                wakeUpTime = offset + jitter noise;
            }
        }

        alarming(wakeUpTime);
    }
}

protected void body()
{
    if (VDM++)
    {
        waitUntil(expected)
    }

    new PeriodicThread(exected from nextTime()).start();

    run operation...
}
```

Every time the body of a periodic thread is executed, it first creates the *next* thread to run, and in VDM-RT puts it into an ALARM state so that it does not until the expected time, or in VDM++ busy-waits until the expected time. This means that periodic iterations can overlap if the operation takes longer than the period. The *nextTime* method performs a calculation based on jitter and delays, using a PRNG (remember a PRNG *nextLong* call can be positive or negative):

```
private long nextTime()
{
    // "expected" was last run time, the next is one "period" away,
    // but this is influenced by jitter as long as it's at least
    // "delay" since "expected".

    long noise = (jitter == 0) ? 0 : PRNG.nextLong() % (jitter + 1);
    long next = SystemClock.getWallTime() + period + noise;

    if (delay > 0 && next - expected < delay) // Too close?
    {
        next = expected + delay;
    }

    return next;
}
```

Naively, the next time will just be "now" plus the period. But if there is a jitter value set, an amount of "noise" is calculated randomly between +jitter and -jitter and this is also added to the next time calculation. Lastly, if the resulting next time is too close to the previous one (less than the "delay" value) then the next value is set to precisely "delay" ticks after the last one was expected.

When *SchedulableThreads* try to invoke operations with a sync guard, they may need to enter a WAITING state until the guard value changes. This uses a *Lock* class as mentioned above. The method to test a sync guard is called "guard", and is called before invoking any operation that has a sync condition defined. The outline of the method is as follows:

```
private void guard() throws ValueException
{
    self.guardLock.lock();
}
```



```
while (true)
{
    synchronized (self)    // So that test and act() are atomic
    {
        // We have to suspend thread swapping round the guard,
        // else we will reschedule another CPU thread while
        // having self locked, and that locks up everything!

        debug("guard TEST");
        setAtomic(true);
        boolean ok = guard.eval();
        setAtomic(false);

        if (ok)
        {
            debug("guard OK");
            act();
            break;        // Out of while loop
        }
    }

    // The guardLock list is signalled by the GuardValueListener
    // and by notifySelf when something changes.

    debug("guard WAIT");
    self.guardLock.block();
    debug("guard WAKE");
}

self.guardLock.unlock();
}
```

The important points are that the guardLock is held on a "self" basis – not per operation. That is because guards often refer to `#act(op)` etc for operations other than the current one, and so all changes to self should be signalled through a Lock held at that level. The other important point is that the test and the call to `act()` to increment `#act` must be made under a Java lock to avoid another thread sneaking in (though this is probably not necessary in the deterministic scheduler, it is still good practice – it was *essential* with the non-deterministic scheduler).

The `self.guardLock` is signalled (to wake up a blocked thread) by a `GuardValueListener`. This implements the `ValueListener` interface, and is associated with `UpdatableValues` that comprise the state which is read by each guard expression. Therefore changes to those values cause the guard to be re-evaluated. The guardLock is also signalled by the `OperationValue`'s `notifySelf` method, which is called whenever any operation's `#req`, `#act` or `#fin` value changes – note that this is rather crude, unlike the `UpdatableValues`, and means that guards wake up for this reason more often than they need to.

Apart from `BUSThread` (which runs specialized code for BUS transfers) the other `SchedulableThread` subclasses all evaluate some sort of expression in a context that they create at the start. This is done in their body methods:

```
@Override
public void body()
{
    try
    {
        DebugLink link = DebugLink.getInstance();
        link.setCPU(CPUValue.vCPU);
        result = expression.eval(ctxt);
    }
    catch (ContextException e)
    {
    }
}
```



```
        setException(e);
        suspendOthers();
        DebugLink.getInstance().stopped(e.ctxt, e.location);
    }
    catch (Exception e)
    {
        while (e instanceof InvocationTargetException)
        {
            e = (Exception)e.getCause();
        }

        setException(e);
        suspendOthers();
    }
    finally
    {
        TransactionValue.commitAll();
    }
}
```

The calls to `setException` causes the main thread to throw an exception at the very end of the evaluation (even if the exception was raised in a different thread). It also reports the exception on `stderr`. The catch clauses that enter the debugger call `suspendOthers`, which sends a signal to the other threads to suspend, trapping them into the debugger too (see above). For exceptions where we have no VDM context information, we just suspend all threads and let the current thread die (which we can't debug, as we have no context).

2.9.1. Comments

The amazing thing about this is that it actually performs quite well :-)

The weakest part of the system is probably the way that history counter changes wake up all guards on the object rather than just those that refer to the history values concerned. The problem is that history values are not `UpdatableValues` and therefore cannot simply have `GuardValueListeners` attached. A related inefficiency is that, while a `GuardValueListener` will only trigger on value updates that are mentioned in a guard, it wakes up all guards on the object because the lock is object-wide.

This may cause problems with debugging because guard tests will be very frequently scheduled, and so control will flip to those threads very often, even when the guard cannot have changed.

The use of scheduler timeslice jitter (and the FP policy in general) does not have as great an affect with VDM-RT as one might imagine. This is because RT threads hardly ever exhaust their timeslice before suspending. Rather, thread suspension is usually caused by the duration that every statement has. This is not the case for VDM++ though.

2.10. vdmj.values

Interface Summary	
ValueListener	Implemented by classes that watch for changes to a Value.

Class Summary	
Value	The parent of all runtime values.
****Value	A value of type ****. There are about 30 of these.
NumericValue	The root of the numeric value types.
ReferenceValue	The root of the value classes which reference another value.
InvariantValue	A ReferenceValue which includes an invariant function.



UpdatableValue	A ReferenceValue in which the referenced value can be changed.
GuardValueListener	A listener for processing updates that affect operation guards
ClassInvariantListner	A listener for processing updates that affect object instance invariants.
NameValuePair	A class to hold a name and a runtime value pair.
NameValuePairList	A list of name/value pairs.
NameValuePairMap	A map of name/values.
Quantifier	A class representing a quantifier.
QuantifierSet	A class representing a set of quantifiers.
State	A class for holding a module's state data.
ValueList	A sequential list of values.
ValueMap	A map of value/values.
ValueSet	A set of values.
BUSValue	A VDM-RT interconnecting BUS.
CPUValue	A VDM-RT CPU.
TransactionValue	A VDM-RT thread-updated value.

The `vdmj.values` class contains a set of value classes to represent runtime values in the interpretation of a specification. They are all subclasses of the abstract `Value` class.

All `Values` implement a set of standard methods to allow them to work correctly with the Java collections framework: `equals`, `hashCode`, `toString`, `clone`.

The `convertTo` method takes a `TCType` argument and is responsible for the dynamic type conversion of values from one type to another, where possible. This tests the `-dtc` settings flag, returning the value unchanged if the flag is set. The method is used when types are “enforced” in a specification, such as when values are passed as typed arguments, or when values are returned from a typed function or operation. The method is specialized in all the `Value` subclasses that generally know how to convert themselves into a small number of related types (eg. conversions between subclasses of `NumericValue`). If a value cannot convert itself, it calls up to its superclass. The `convertTo` at the root of the `Value` hierarchy deals with common complex cases: converting to a union (iteratively trying to convert to each type); converting to a parameter type (looking up the parameter’s name to get its actual type); converting to optional types (convert to the underlying type); and converting to a named type (convert to the underlying type, then wrap with an `InvariantValue` to enforce any type invariant).

A `ReferenceValue` is an abstract class that contains a value, and delegates all operations to that contained value. The concrete subclasses are `InvariantValue`, which includes an invariant `FunctionValue` as well as the value; and `UpdatableValue`, which has a `set` method capable of changing the value referenced (and all its methods are synchronized to allow for safe access to the changed value from other threads). When an `UpdatableValue` is created, it is passed a list of `ValueListeners` (optionally), which are called back when the `set` method is called. This is used to invoke class or state invariant functions when values are changed, and to trigger guards in VDM++ and VDM-RT.

`FunctionValues` and `OperationValues` are slightly special in that they can evaluate themselves. This includes the evaluation/checking of pre and postconditions, and in the case of recursive functions, the evaluation of any measure function which is defined. The latter maintains a per-thread stack of measure values in the `FunctionValue`, and successive calls must be “less than” the previous one, which is determined using the `compareTo` method.

Boolean expressions can work with *undefined* values, represented by and `UndefinedValue`. The “undefined” keyword (which is non-standard) introduces an undefined value, and boolean operators deal with this in accordance with the rules of LPF¹. For example, “true or undefined” is true, but “true and undefined” is undefined. Roughly, the undefined boolean value means we’re not sure whether the

¹ The “Logic of Partial Functions”, which is the formal logic underlying VDM.

argument is true or false; sometimes the result of an operation with such an argument is well defined, sometimes not. So “exists x in set {true, undefined} & x” is true, because there definitely is one value that is true, even if another value is undefined. But “forall x in set {true, undefined} & x” is undefined, because one of the values is not definitely true. This behaviour is only supported for boolean expressions. If an undefined value is used in any other context, such as a numerical calculation, the interpreter will abort immediately. For example, “123 + undefined” produces “Error 4089: Can't get real value of undefined”.

A State class holds a VDM-SL module's state, which is held in an UpdatableValue which contains a RecordValue – being the “mk_Sigma” record value that can be considered to hold the Sigma state. The class implements ValueListener, which means that any changes to the UpdatableValues which comprise the module state will automatically call the changedValue method of State to check whether any state invariant has been violated. The class includes a flag which suppresses the invariant check while the state is being initialized.

A GuardValueListener implements ValueListener, holding an ObjectValue to signal when it received a value update. This is used to identify when updates are made to variables which are identified in operation permission guards in VDM++ and VDM-RT. The changedValue method calls the signal method of a Lock object in the ObjectValue, which is also waited for by operation guards.

A ClassInvariantListener also implements ValueListener, holding an invariant OperationValue which can be executed in the event that any of the dependent variables changes value. This is used to implement object instance invariants. The class includes a boolean flag, called doInvariantChecks, which can be used to disable checks – for example during a class construction, or during an atomic statement.

Most value classes are immutable, in the sense that they are constructed with an underlying value that is held in a (public) final field of the class, and never changes. The only exception to this is the UpdatableValue, whose referenced value can be modified by the set method – note this changes the immutable value referenced; it does not change the value of the referenced object.

UpdatableValues are used for “state” values (instance variables, module state and “dcl” values that can be changed in an assignment). They are often initialized with structured immutable values, so values implement a method called “getUpdatable” to create UpdatableValue versions of themselves and their sub-values.

A Value can be held together with a TCNameToken in a NameValuePair, and such pairs can be collected into a list or a map for convenience. For example, the members of an ObjectValue are held in a NameValuePairMap.

Internally, many value classes have basic Java types at their heart. The basic values underlying SetValues, SeqValues and MapValues are ValueSets, ValueLists, and ValueMaps respectively. The basic value of a Value can be extracted using one of several methods, like realValue or functionValue. If the Value concerned cannot be converted to the basic type sought, a ValueException is thrown. The basic underlying values are used in the eval method of expressions to (say) actually perform arithmetic.

To ensure deterministic behaviour of specifications, two value types must enforce an ordering on their contents where none is strictly necessary: sets and maps. If this was not done, expressions which iterate over these values could produce different results for different executions. Therefore, when a SetValue is constructed, its underlying ValueSet contents are sorted; similarly, a MapValue is based on a Java TreeMap which keeps the contents in a sorted order.

All Value classes implement the Comparable interface and the compareTo method. InvariantValues and RecordValues can use ordering and equality functions defined in their type definition to implement these methods; numeric values have a natural ordering; other values implement an arbitrary but predictable order.

A CPUValue represents a CPU declaration in a VDM-RT system class. Creating such a value also creates a CPUResource, which registers itself with the resource scheduler. A BUSValue similarly represents a BUS connection in VDM-RT, creating and registering a BUSResource when it is constructed. A BUS comprises a link between a set of CPUs.

A BUSValue has two important methods: transmit and reply. These delegate though to the BUSResource, which creates an AsyncThread to handle a request, and signals a Holder with a reply.



In VDM-RT, when multiple threads access a shared variable – eg. a piece of state information in an object – updates to the variable value are only visible *outside* the current thread after the next timestep (while changes are obviously visible inside the modifying thread). A value cannot be modified by two threads concurrently in VDM-RT (this is a runtime error). This processing is managed by a TransactionValue class. This is a subclass of UpdatableValue, where the set method is overridden to write the new value to a local field rather than to the referenced value from the parent. The various retrieve methods (intValue, booleanValue etc) are overridden to use the new value for the thread that changed it, and the parent value otherwise. An additional commit method actually sets the parent value to the new value. The commit method is called during a time step, making the per-thread value visible to other threads.

Transactional variables are very new, and by default the commit system is disabled. It can be enabled by a `vdmj.rt.duration_transactions` property setting.

2.10.1. Comments

It seemed sensible to have all Values contain some sort of primitive value, including sets, sequences and maps, rather than having the Value classes extend the Java collection framework directly. That allows the Value types to be in a "VDM" hierarchy, but it leads to the confusing situation where a MapValue contains a ValueMap – the former extends Value, the latter extends TreeMap<Value,Value>. The ValueXXX classes should only be used internally – ie. they should never be returned from an eval method (they can't be, as they're not Value subtypes).

There is a lot more discussion about VDM-RT and time handling in section 2.14.

2.11. vdmj.plugins

Class Summary	
VDMJ	The “main” entry point for the plugin based console.
LifeCycle	Defines the order in which a spec is processed.
PluginRegistry	A register of plugins in the system.
EventHub	A publish/subscribe hub for communicating events to plugins.
EventListener	An interface implemented by event subscribers.
AnalysisPlugin	An abstract base class for all analysis plugins.
AnalysisCommand	An abstract base class for commands defined by plugins.
AnalysisEvent	An abstract base class for events sent to plugins.
PluginConsole	Methods to allow plugins to write to the console
CommandReader	The interactive command line reader for plugins.
HelpList	An ordered list of help strings for AnalysisCommands.
analyses.*	Standard plugins for AST, TC, IN, PO and CMD.
commands.*	Standard commands provided by AST, TC, PO and CMD.
events.*	Standard events issued by VDMJ for plugins.

The `vdmj.plugins` package contains a flexible approach to reading the console and coordinating actions with various analyses. This is done via “plugins” which perform arbitrary language and control actions, and which can be provided by end users to extend the behaviour of the suite.

The original `vdmj.command` classes were hard-wired to invoke the analyses provided by VDMJ (section 2.5). But to be more flexible, we ideally want end users to be able to provide their own analyses, and have these seamlessly fit together with the standard ones. These new analyses might want to have their own Java command line options, and provide their own console commands too, for example.



To solve this, `vdmj.plugins` re-packages the standard analyses (AST, TC, IN, PO) in a form that can also be extended by end users. The means by which (say) the “-w” flag is handled to suppress warnings is the same that a user provided plugin might handle a “-widget” option; and the means by which the “print” command is made available by the CMD plugin is the same that a user provided plugin might use to provide a “status” command (complete with “usage” and “help” for these settings).

The core of the plugin design is the `AnalysisPlugin` class. This abstract base is used by all analysis plugins and is extended by the `ASTPlugin`, `TCPlugin`, `INPlugin`, `CTPlugin` and `POPlugin` classes to provide the parser, type checker, interpreter, combinatorial testing and POG features. In addition, a `CMDPlugin` coordinates the command line reader.

Each plugin must define a static factory method, which is used to create an instance:

```
public static AnalysisPlugin factory(Dialect dialect) throws Exception
```

The factory can create a plugin that is specialized for the particular dialect passed, or it can simply instantiate a common class for all dialects. Plugins must define a `getName`, a `getPriority` and an `init` method. They may optionally define `processArgs`, `usage`, `getCommand` and `getCommandHelp` methods that allow it to process private command line arguments (like “-widget”) and provide usage information for them, or provide console commands (like “status”) with help information.

Plugins also typically implement the `EventListener` interface. This requires them to add a `handleEvent` method, which is called if the plugin also registers for particular events via the `EventHub`. This registration is expected to be performed by the plugin’s `init` method. Events are sent to all the plugins (if they register for them) during the phases of a parse/check/init of a specification by the `LifeCycle`:

- `CheckPrepareEvent` is sent at the start, before anything else is done, which allows the plugins to initialize themselves in preparation for the following events.
- `CheckSyntaxEvent` is sent to perform a syntax analysis. This is primarily used by the `ASTPlugin` of course, but any plugin can register for the event.
- `CheckTypeEvent` is sent to perform type checking of the specification. This is primarily used by the `TCPlugin`, but can be used by other plugins if they want to use TC information. For example, the `POPlugin` uses this event to prepare its own class mapped tree of the specification, based on the checked TC classes.
- `CheckCompleteEvent` is sent after type checking and is typically used to do things that require the interpreter. So the `INPlugin` uses this event to initialize the specification runtime, for example. The `POPlugin` uses it to generate “one shot” POs when the user gives the “-p” flag.
- `StartConsoleEvent` is sent to start an interactive console, if needed. This is used by the `CMDPlugin` to run the `CommandReader`, if the `-i` option is set.
- A `ShutdownEvent` is finally sent when the user closes the console session, or the execution of a “-e <exp>” is finished (ie. this is always sent, no matter how the session ends).

The `handleEvent` method for these events returns a `List<VDMMessage>` which can contain errors or warnings that the plugin wants to raise. If any `VDMErrors` are returned, the main processing stops at that point and sends a `CheckFailedEvent` to the plugins, which can react to the failure.

Event objects themselves have `getProperty` and `setProperty` methods, which allows related plugins to pass information. The same `AnalysisEvent` object is passed to each plugin in turn, so properties set by (say) the type checker could be read by (say) the POG plugin. Plugins are passed events in a strict order, defined by their `getPriority` values: AST, TC, IN, PO, CT, CMD, followed by any user defined plugins in the order of the “`vdmj.plugins`” resource(s).

Assuming the specification parses and type checks cleanly, and the user gives the “-i” option, the `CommandReader` is started by the `CMDPlugin`, which prompts the user for commands to execute. These requests are passed to the plugins, in order, via the `getCommand` method. The plugins can return an `AnalysisCommand` object, which has a `run` method to do whatever they want it to do. All of the standard commands are implemented this way, and serve as a good example of how to write new ones. Any plugin can write text messages to the console using various methods from the `PluginConsole` class.



The main processing is defined by the LifeCycle class. This defines a Java “run” method that sends out events via the EventHub to be processed by the plugins, though it *has no idea* what the plugins do. The LifeCycle itself is invoked by the “main” method of the com.fujitsu.vdmj.VDMJ class.

2.11.1. Comments

The design of VDMJ plugins is very similar to that of the plugin system used by the LSP language server in the VDMJ suite. It cannot be identical, however, since the LSP server is driven by, and produces JSON messages for the LSP protocol, and the lifecycle is different. But the hope is that it would be relatively easy to produce two plugins for a user analysis, with thin wrappers around a common core of functionality.

2.12. vdmj.messages

Class Summary	
VDMMessage	The root of all reported messages.
VDMError	A VDM error message.
VDMWarning	A VDM warning message.
Console	A class to provide System.in/out with a charset encoded wrapper.
Redirector	An abstract class to redirect output.
StdoutRedirector	A class to redirect standard output to a debugger.
StderrRedirector	A class to redirect standard error to a debugger.
RTLogger	A VDM-RT class for logging thread, CPU and BUS activity.
RTValidator	A VDM-RT class for verifying logs against validation conjectures.

Exception Summary	
MessageException	An exception to carry a textual error message.
NumberedException	An exception to carry a number and text information.
LocatedException	An exception to carry number, text and location information.

The vdmj.messages package contains classes and exceptions to hold and display error and warning messages. Lists of VDMMessage values are returned from the TypeChecker and Reader classes. Similarly, internal exceptions that carry message numbers and position information are all subclasses of NumberedException.

The Console class manages output to stdout/stderr, in particular managing the character set to be used. The Redirector and its concrete subclasses are used to redirect or copy stdout and stderr to a debugger (see section 2.18).

The RTLogger is used by VDM-RT specifications to write loggable events – such that the timing diagram can be analysed using Overture's real-time log viewer. The class caches events in memory, occasionally flushing them out to a given PrintStream. By default this is the Console. It is also possible to turn logging off (the default state), which causes log events to be discarded.

VDM-RT specifications can contain annotations to identify validation conjectures, for example that certain pairs of operation calls must occur within a certain time window. The RTValidator class looks for these annotations and processes log files written by the RTLogger to determine whether the conjectures have been met.



2.13. vdmj.debug

Class Summary	
DebugLink	The interface for debug control.
DebugExecutor	Process debugger commands in the context of one frame.
DebugParser	Parse debugger commands.
DebugReason	Reason code for the completion of a thread.
DebugType	Enum of debug command types
DebugCommand	A debug command type, plus Object payload.
TraceCallback	An interface called by tracepoints.
ConsoleDebugExecutor	The console implementation of DebugExecutor
ConsoleDebugLink	The console implementation of DebugLink
ConsoleDebugReader	The console debugger main class.
BreakpointReader	A class to read breakpoint related commands from the console.

The `vdmj.debug` package contains classes that implement the `DebugLink` interface. This allows VDMJ to load a set of specifications and evaluate/debug them under the remote control of another process – usually a GUI IDE such as Eclipse or VS Code, but by default from the console.

Note that VDMJ does not define the debugger user interface. It only defines the `DebugLink` interface and the threading model associated with that. It is then up to developers to implement something that follows the `DebugLink` interface. The default `ConsoleDebugLink` is the simplest implementation, which just uses the console.

The abstract `DebugLink` class provides the means to create and find a singleton instance of the class that implements it. The concrete implementing class should be set in the property `vdmj.debug.link_class`. If this is not set, the default `ConsoleDebugLink` is instantiated.

Methods of the `DebugLink` implementation will be called when the debugged process reaches various significant points:

- *newThread* is called when a new thread is created by the process.
- *stopped* is called when execution is stopped, for example because another thread has hit a breakpoint.
- *breakpoint* called when a breakpoint is reached.
- *tracepoint* is called when a tracepoint is reached.
- *complete* is called when a thread finishes or fails with a `ContextException`.
- *getExecutor* is called when a new `DebugExecutor` instance is required.

Note that each of these calls is made from a thread context that matches the call. So if a new thread is created, *newThread* is called from the context of that new thread. This means that the `DebugLink` implementation must be thread-aware.

Typically, in addition to the `DebugLink`, other classes must be implemented to provide a complete debugging environment. In the default implementation, the most important of these is the `ConsoleDebugReader`. This extends the Java `Thread` class as well as implementing the `TraceCallback` interface. A running instance of the thread is created from the command line execution environment when the user evaluates something, like “`print func(args)`”.

The *run* method of the `ConsoleDebugReader` thread loops, calling the `ConsoleDebugLink` instance, waiting for at least one thread to have stopped. When a thread stops, the loop calls out to a method that prints (to the console) the thread name and location at which it stopped. Then it waits for a command to be entered at the console. The command typed is parsed by `DebugParser` to produce one of a set of *standard* `DebugCommands` (which include a `DebugType` and an object payload).



A couple of commands can be dealt with locally: the *threads* command obtains a list of current threads from the DebugLink and lists them on the console; and the *thread* command selects one of the threads listed as the current thread for further debug commands to be sent to.

But most debug commands are sent through to the DebugLink. The *stop* and *quit* command (synonyms) result in a call to DebugLink *killThreads*, but most commands are sent to the *sendCommand* method, which returns a DebugCommand result. As well as carrying a payload that can be printed, the return result from *sendCommand* indicates whether execution should continue as a result (for example, a *step* or *continue* command would do this, but a *stack* command would not), and the *resumeThreads* method must be called if this is indicated.

The DebugLink will ask for a DebugExecutor for each new stack frame context when it stops. The *getExecutor* method is passed the LexLocation of the execution and the frame Context. The default implementation, ConsoleDebugExecutor, implements each of the supported commands (typically) by using the Context passed and the Interpreter instance to find the information required or change the runtime state as required. The return value is one of a set of convenience DebugCommand values that have no payload, or information that the ConsoleDebugReader can interpret in the payload (usually a string for display, such as a stack trace).

The *TraceCallback* interface is implemented by ConsoleDebugReader, and prints trace information to the console. The ConsoleDebugLink *tracepoint* method calls the interface, passing the Tracepoint and the Context of the execution.

A BreakpointReader is used to handle break, trace, list, remove and catch commands, since these can be used from the command prompt or when stopped at a breakpoint.

2.13.1. Comments

This interface and thread model has proved flexible enough to implement both the DBGP protocol [6], and the DAP protocol [10], though I suspect that it could be simplified a little and still work perfectly well. The complexity is within the implementations of the DebugLink and DebugExecutor classes, where particular attention has to be paid to which thread is calling the code.

2.14. vdmj.config

Class Summary	
Properties	A value to provide access to the vdmj.properties file.

This is a simple class with static fields, each representing (and updated with) values in the vdmj.properties file. The class extends ConfigBase, which contains reflection calls to lookup field names in the properties file (if any) and update the default static values with the property read. There is a vdmj.properties file included with VDMJ which just contains the following default values:

```
#
# Settings for VDMJ. These override defaults in the code.
#

# The tab stop for source files (default 4)
vdmj.parser.tabstop = 4

# Nesting of block comments: 0=support, 1=warn, 2=err, 3=ignore (default 3)
vdmj.parser.comment_nesting = 3

# External readers .suffix>=<class> pairs (default null)
vdmj.parser.external_readers = null

# Enable merging of adjacent line comments into a single block comment.
vdmj.parser.merge_comments = false
```



```
# The package list for annotation classes to load.
# (default "com.fujitsu.vdmj.ast.annotations;annotations.ast")
vdmj.annotations.packages =
com.fujitsu.vdmj.ast.annotations;annotations.ast

# Enable annotation debugging (default false)
vdmj.annotations.debug = false

# An alternative search path for the ClassMapper (default null)
# vdmj.mapping.search_path = null

# Skip the check for mutually-recursive function calls (default false)
vdmj.tc.skip_recursive_check = false

# Skip the check for definition dependency cycles (default false)
vdmj.tc.skip_cyclic_check = false

# The maximum TC errors reported before "Too many errors" (default 100)
vdmj.tc.max_errors = 100

# The maximum expansions for "+" and "*" trace patterns (default 5)
vdmj.traces.max_repeats = 5

# Serialize the system state between trace tests (default false)
vdmj.traces.save_state = false

# The size below which trace function args are expanded (default 50)
vdmj.traces.max_arg_length = 50

# The default timeslice (statements executed) for FCFS policy (default 10)
vdmj.scheduler.fcfs_timeslice = 10

# The vCPU/vBUS timeslice (default 10000)
vdmj.scheduler.virtual_timeslice = 10000

# The timeslice variation (+/- jitter ticks, default 0)
vdmj.scheduler.jitter = 0

# The default duration for RT statements (default 2)
vdmj.rt.duration_default = 2

# Enable transactional variable updates (default false)
vdmj.rt.duration_transactions = false

# Enable InstVarChange RT log entries (default false)
vdmj.rt.log_instvarchanges = false

# Maximum period thread overlaps per object (default 20 - zero mean off)
vdmj.rt.max_periodic_overlaps = 20

# Enable extra diagnostics for guards etc. (default false)
vdmj.rt.diags_guards = false

# Enable extra RT log diagnostics for timesteps (default false)
vdmj.rt.diags_timestep = false

# The packages for command plugins to load from (default "plugins")
vdmj.cmd.plugin_packages = plugins

# A list of user defined VDMJ plugins (default null)
# vdmj.plugins = <fully qualified class CSV list>
```



```
# The class name for the DebugLink (default null)
# vdmj.debug.link_class = null

# The set size limit for power set expressions
vdmj.in.powerset_limit = 30

# The type size limit for type bind expansions
vdmj.in.typebind_limit = 100000

# The max stack to dump via println(Throwable) (default 1, zero means all)
vdmj.diag.max_stack = 1;
```

2.15. vdmj.util

Class Summary	
Utils	A utility class.
Base64	A class to encode/decode base64 strings.
Delegate	A class to hold a native delegate Java object.
DependencyOrder	A class to help with arbitrary dependency orderings
KPermutor	A class to enumerate (n k) permutations
KDuplicatePermutor	A class to enumerate (n k) permutations, with duplicates
KCombinator	A class to enumerate (n k) combinations
Selector	A class to produce every selection of one value from an array of limits.
Progress	An interface for tracking progress of long operations
NullProgress	A default implementation of Progress

Utils is a small utility class. It just defines a few methods for printing out a comma separated list of arbitrary types held in a Java List<T>, and a few other methods.

Base64 does what you would expect. It is used by the DBGp protocol.

The Delegate class is used to lookup a Java class of the same name as a given VDM++ or VDM-RT class or VDM-SL module, and hold a reference to it. If such a class is found, and the VDMJ specification encounters an "is not yet specified" expression or statement, then the call is delegated to a similarly named method in the native delegate object. This is how the IO, MATH and VDMUtils classes/modules are implemented. See section 3.1.

DependencyOrder can be given a list of module, classes or definitions, and produces a graph of the dependencies of each on the others. This can be used to produce a "dot" format file or to drive a topological sort of the elements of the list.

The permutor, combinator and selector classes are used to generate various sorts of combinations, which are used in various places to "try all possibilities", like the generation of power sets. Note that these implement the Java Iterator interface, so they can be used directly in "for" loops.

The Progress interface defines methods which very slow operations can use to report progress and test for a user cancellation. This is typically used for user-visible commands. The NullProgress implementation can be passed where a Progress object is needed, but none is provided.

3. External Interfaces

3.1. The Native Call Interface

Parts of a VDM specification may wish to call out to native Java code, for example to compute mathematical library functions or to perform IO operations. The external calling mechanism in VDMJ uses the "is not yet specified" statement and expression to achieve this.

When a non-specified statement or expression is encountered, VDMJ determines whether the class or module containing it has a Java "delegate". A delegate is a Java object which has methods of the same name as the non-specified VDM operation or function, and to which the evaluation can be delegated, returning a real result rather than a runtime exception.

If there is no delegate, and this is the first time a non-specified statement or expression has been encountered in the current module or class, then VDMJ looks through the Java CLASSPATH for a Java class of the same name as the name of the module or class (with underscores turned into package separators). So for example, a module or class called `org_xyz_Handler` would look for a class in the `org.xyz` package called `Handler`. If such a Java class is found, an instance of the class is created, and associated with the Module, the `ClassDefinition` or the `ObjectValue`, depending on whether the operation or function being invoked is static.

The delegate object is thereafter associated with the module, class or object in VDM, and so state values written inside the Java object will be retained. Calls to the "is not yet implemented" operations or functions are passed to the delegate, with argument values passed as VDMJ Values. Similarly, the return value of the Java method must be a VDMJ Value.

It is a runtime error to have delegate methods which do not take Value arguments or return Value. It is also a runtime error to have non-static operations/functions delegate to static Java methods. You can only distinguish delegated overloaded names by the number of Value parameters (ie. you cannot distinguish overloaded members by the actual types of the parameters, as they are all Value in Java).

The delegate discovery call is only made once. Thereafter, if there is no delegate, the "is not yet specified" statement or expression will raise a normal `ContextException`.

Note that a class or module with a simple name like `MATH` or `IO` must have a delegate Java class in the default Java package. The `MATH`, `IO` and `VDMUtil` standard library functions in VDMJ are implemented using the same delegate scheme. It is possible to delegate from flat VDM-SL specifications, but the Java class has to be called `DEFAULT` (which is the VDM module name for flat specifications).

If Java methods in the delegate need to construct VDM Values, a `ValueFactory` provides a set of methods to achieve this. The factory methods are all static:

```
public static BooleanValue mkBool(boolean b)
public static CharacterValue mkChar(char c)
public static IntegerValue mkInt(long i)
public static NaturalValue mkNat(long n) throws Exception
public static NaturalOneValue mkNat1(long n) throws Exception
public static RationalValue mkRat(long p, long q) throws Exception
public static RationalValue mkRat(double d) throws Exception
public static RealValue mkReal(double d) throws Exception
public static NilValue mkNil()
public static QuoteValue mkQuote(String q)
public static SeqValue mkSeq(Value ...args)
public static SetValue mkSet(Value ...args) throws ValueException
public static TupleValue mkTuple(Value ...args)
public static TokenValue mkToken(Value arg)
public static RecordValue mkRecord(String module, String name,
                                   Value ...args) throws ValueException
public static InvariantValue mkInvariant(String module, String name,
                                         Value x) throws ValueException
```

3.2. The Remote Control Interface

It is sometimes desirable to have VDM specifications "controlled" by external programs. To enable this, VDMJ provides a RemoteControl interface which must be implemented by external programs wishing to take control of a specification.

The interface only defines one method:

```
public interface RemoteControl
{
    public void run(RemoteInterpreter interpreter) throws Exception;
}
```

The class implementing this interface is identified to VDMJ using the `-remote` command line argument. If such an option is given, the specification is parsed and checked as normal, but instead of entering the interpreter, an instance of the remote class is instantiated (default constructor) and the `run` method is called, passing a `RemoteInterpreter` object. The class is responsible for implementing the `run` method such that control does not return until the program is complete. The `RemoteInterpreter` passed has three methods which can be called to interact with the specification: `execute`, `valueExecute` and `init`. The first takes a string argument and evaluates it, returning a string value; the second takes a string argument and returns a VDMJ Value; the third re-initializes the interpreter.

Any exceptions (eg. runtime `ContextExceptions`) raised by the interpreter as a result of calling the methods on `RemoteInterpreter` will be returned to the controlling class.

One possible use of the remote control interface is with the JUnit testing framework to test VDM specifications. For example, a `run` method could be written as follows:

```
public void run(RemoteInterpreter i) throws Exception
{
    TestClass.init(i);
    TestRunner.run(TestClass.class);
}
```

This is a fairly crude way of invoking JUnit's text test runner, identifying a `TestClass` containing a set of tests which JUnit will iterate through, presumably calling the `valueExecute` method of the `RemoteInterpreter`, and asserting the value of the results or exceptions returned. The JUnit `setUp` method of `TestClass` can be used to initialize the interpreter between tests, if that is required.

Similarly, the `RemoteSession` class in the default package implements the `RemoteControl` interface to provide a simple command-line read/eval loop for expressions. This is used by the Overture GUI to launch a debugging *session* rather than making individual evaluations.

```
public class RemoteSession implements RemoteControl
{
    public void run(RemoteInterpreter interpreter)
    {
        open a console...

        while (carryOn)
        {
            read a line...

            if (line.equals("init"))
            {
                interpreter.init();
            }
            else
            {
                String output = interpreter.execute(line);
                System.out.println(line + " = " + output);
            }
        }
    }
}
```

3.3. The Remote Simulation Interface

VDM-RT specifications are used to simulate systems of CPUs connected by buses. The RT logging information produced can be used to analyse the execution of the system after the event, but it is sometimes useful to have monitoring components “in the loop” to monitor the state of the system as it evolves. This is the purpose of the abstract `RemoteSimulation` class.

A concrete `RemoteSimulation` must implement two methods: `setup` and `step`.

The `setup` method is passed an `ASTClassList` early in the simulation. The intention is to allow the class to change various system parameters that influence the evaluation of the model. Supporting this, there are “getter” and “setter” methods provided, either for reading/overwriting constants in particular fields of the AST, or for setting variables in the “system” class.

The `step` method is called at the start and at intervals during the model evaluation. It is passed the current system clock (`SystemClock.getWallTime()`) and it returns the time at which it should next be called. The `step` can make use of the same getter and setter methods to read or change the running system.

VDMJ includes an example simulation class, as follows:

```
/**
 * This simple example is designed to run with the sporadic.vdmrt test
 * spec in examples/simulation/src/test/resources. It demonstrates setting
 * parameters and changing the value of runtime system variables during
 * a simulation.
 *
 * Execute in VDMJ by adding the project to the classpath and passing:
 *
 *     -simulation simulation.TestSimulation
 *
 * Then execute "new Test().test()" as usual.
 */
public class TestSimulation extends RemoteSimulation
{
    @Override
    public void setup(ASTClassList classes)
    {
        setParameter(classes, "A", "MIN", 50);
        System.out.println("Set MIN to 50");
        setParameter(classes, "A", "MAX", 100);
        System.out.println("Set MAX to 100");
    }

    @Override
    public long step(long time)
    {
        try
        {
            Long last = getSystemIntegerValue("obj1", "last");
            System.out.println("Last = " + last);

            setSystemValue("obj1", "last", last + 1);
            last = getSystemIntegerValue("obj1", "last");
            System.out.println("Updated = " + last);
        }
        catch (ValueException e)
        {
            System.err.println(e);
        }

        return time + 1000;
    }
}
```



The output should be like this:

```
> p new Test().test()
Last = 0
Updated = 1
Last = 1
Updated = 2
Last = 2
Updated = 3
Last = 3
Updated = 4
Last = 4
Updated = 5
Last = 5
Updated = 6
Last = 6
Updated = 7
= ()
Executed in 0.057 secs.
>
```